SISTEMAS DE ENGENHARIA / GUIAS DE UTILIZAÇÃO

ArquitecturaRepo

Manual completo de utilização

VERSÃO 1.4 / PORTUGUÊS

Oito capítulos completos: descoberta, instalação, adoção, operação, mudanças, recuperação, aceitação e um exemplo guiado de produto.

MATÍAS ACUÑA · NOVEMBRO DE 2026

Documentação verificada com o código-fonte do ArquitecturaRepo.

Índice geral

00 — Antes de começar **5**
Neste capítulo . . . 5
1. O que está a instalar e para quê . . . 5
2. Quem decide cada questão . . . 6
3. Informação que deve permanecer acessível . . . 6
4. Como utilizar este manual . . . 7
5. Preparação das decisões que só o CLIENT pode tomar . . . 7
6. Alcance, limites e versões . . . 8
7. Um percurso de ponta a ponta . . . 8
8. Decisões frequentes e responsabilidades . . . 9
9. Ficha de arranque de uma página . . . 9
10. Como avaliar se o sistema está a ajudar . . . 10
Fontes e âmbito . . . 10

01 — Instalação e configuração **12**
Neste capítulo . . . 12
1. Ambiente e contas . . . 12
2. Starter, fork e aplicação: três lugares diferentes . . . 13
3. Verificar caminhos antes de escrever . . . 13
4. Escolher um perfil proporcional . . . 14
5. Criar a camada de controlo . . . 14
6. Validar sem inventar aceitação . . . 15
7. A próxima ação é discovery . . . 15
8. Antes de repetir uma instalação . . . 16
9. Passar do scaffold à adoção real . . . 16
10. Conexões, permissões e privacidade . . . 17
11. Diagnóstico seguro . . . 17
Fontes e âmbito . . . 18

02 — O Planner e a adoção do projeto **19**
Neste capítulo . . . 19
1. Uma conversa sobre o projeto, não sobre a ferramenta . . . 19
2. Preparar um espaço de trabalho . . . 20
3. Discovery: fazer as perguntas corretas . . . 20
4. Baseline de produto e plano de entrega . . . 21
5. Adoção: relacionar starter e aplicação . . . 21
6. O exame decisivo de um contexto fresco . . . 22
7. Primeira campanha autorizada . . . 22
8. Exemplo de sessão inicial com o Planner . . . 23
9. Um baseline confirmado pelo CLIENT . . . 23
10. Exemplo de primeira campanha . . . 24
Fontes e âmbito . . . 24

03 — Operação diária **25**
Neste capítulo . . . 25
1. Preparar uma sessão sem recomeçar do zero . . . 25
2. Converter uma intenção numa capacidade verificável . . . 26
3. O trabalho contínuo do Controller . . . 26

4. Autorizações e efeitos externos . . . 26
5. Como ler criticamente um relatório de execução . . . 27
6. Git e continuidade entre contextos . . . 28
7. Gestão de recursos, artefactos e espaço . . . 28
8. Coordenação, concorrência e intervenções do CLIENT . . . 28
9. Revisão independente e candidatos exatos . . . 29
10. Uma rotina curta e reutilizável . . . 29
Fontes e âmbito . . . 30

04 — Solicitar mudanças e novas capacidades 31
Neste capítulo . . . 31
1. Descrever o resultado que se quer obter . . . 31
2. Distinguir novo requisito, bug e refactor . . . 32
3. Um formato simples para pedir alterações . . . 32
4. Prioridades, orçamento e critical path . . . 33
5. Alterações que afetam dados e segurança . . . 33
6. Feedback sobre UX e aceitação humana . . . 34
7. Devolução de defeitos ao mesmo Controller . . . 34
8. Como evitar campanhas e políticas duplicadas . . . 35
9. Exemplos de pedidos bem delimitados . . . 35
10. Confirmar que a alteração ficou atual . . . 36
Fontes e âmbito . . . 36

05 — Erros, incidentes e recuperação 37
Neste capítulo . . . 37
1. Preservar a evidência antes de reparar . . . 37
2. Falhas do ambiente, contas e permissões . . . 38
3. Quando a camada de controlo não valida . . . 38
4. Git não é uma forma de apagar problemas . . . 39
5. Repetição de defeitos e causas sistémicas . . . 39
6. Segurança, privacidade e direitos externos . . . 40
7. Provar a saída de um incidente . . . 40
8. Três cenários de recuperação . . . 41
9. Operações sensíveis e limites de autoridade . . . 41
10. O que uma validação verde não demonstra . . . 42
11. Feedback e dados locais do starter . . . 42
Fontes e âmbito . . . 43

06 — Encerramento de capacidades e aceitação 44
Neste capítulo . . . 44
1. A capacidade é um resultado de produto . . . 44
2. Cinco componentes do encerramento . . . 45
3. Candidato exato como unidade de revisão . . . 45
4. Evidência insuficiente sem campanhas artificiais . . . 46
5. Aceitar, retomar ou mudar o objetivo . . . 46
6. Revisões periódicas e regra de paragem . . . 47
7. Registo de encerramento auditável . . . 47
8. Exemplo de rejeição e correção . . . 48
9. Produto utilizável e framework verde . . . 48
10. O que fazer depois do encerramento . . . 49
Fontes e âmbito . . . 49

07 — Exemplo guiado: aplicação de gestão de um clube **50**
Neste capítulo 50
1. O produto que o clube pretende construir 50
2. Descobrir as questões que mudam o resultado 51
3. Instalar o controlo sem confundi-lo com a aplicação 51
4. Primeira capacidade: gestão de associados 52
5. Do Planner a I3: um pedido operacional 52
6. O que testar antes de aceitar 53
7. Segunda capacidade: mensalidades e pagamentos 54
8. Avaliar todo o exemplo 54
9. Primeiro ciclo com dados fictícios 55
10. Registar e encaminhar um defeito 55
11. As dependências do segundo ciclo 56
12. Exercício final de recuperação 56
Fontes e âmbito 57

00 — Antes de começar

ArquitecturaRepo · Manual completo de utilização · Edição v1.4 · Português

> O ArquitecturaRepo é uma arquitetura de coordenação de desenvolvimento, não um programador automático nem uma aplicação pronta. Ajuda a manter decisões de produto, responsabilidades, evidências e estado atual reconstruíveis mesmo quando uma conversa, ferramenta ou agente deixa de estar disponível. A edição pública v1.4 tem limites de suporte precisos: consulte o guia público atual e as notas de lançamento antes de executar comandos sensíveis à versão.

Neste capítulo

1. O que está a instalar e para quê
2. Quem decide cada questão
3. Informação que deve permanecer acessível
4. Como utilizar este manual
5. Preparação das decisões que só o CLIENT pode tomar
6. Alcance, limites e versões
7. Um percurso de ponta a ponta
8. Decisões frequentes e responsabilidades
9. Ficha de arranque de uma página
10. Como avaliar se o sistema está a ajudar

1. O que está a instalar e para quê

Imagine uma pessoa a desenvolver um produto com ajuda de um chat de programação. Num dia, o assistente cria código, sugere uma arquitetura e resolve um problema. No dia seguinte, a conversa termina, entra outro agente e parte das decisões essenciais perdeu-se entre mensagens. Pode existir muito código, mas ninguém consegue explicar com segurança qual é o resultado pretendido, que ficheiros são os atuais ou porque se escolheu determinada abordagem.

O ArquitecturaRepo procura resolver essa fragilidade. A fonte de verdade fica no repositório e nas superfícies canónicas autorizadas, com referências para requisitos, Product State, responsabilidades, campanhas de engenharia, alterações, testes e evidências. O chat continua útil como interface de colaboração, mas deixa de ser o único local onde existe a história necessária para continuar o trabalho.

Uma instalação inicial acrescenta uma camada `.agentic-sdlc/` ao projeto de destino quando esse projeto não possui já governação canónica incompatível. Essa camada contém contratos, modelos de estado, documentação e ferramentas de validação. Não acrescenta automaticamente uma interface, uma API, um esquema de produção ou uma conta de inteligência artificial. Também não dá autorização para consumir serviços pagos, publicar conteúdo ou manipular dados reais de terceiros.

O valor esperado não é aumentar o número de papéis ou relatórios, mas reduzir três custos concretos: voltar a explicar decisões, perder a identidade do trabalho atual e aceitar resultados sem evidência suficiente. Se a estrutura consumir mais esforço do que o risco que evita, deve ser simplificada dentro dos limites aplicáveis.

2. Quem decide cada questão

O sistema distingue autoridades. O CLIENT é a pessoa que quer o produto: decide necessidades, prioridades, preferências, restrições e efeitos externos reservados. Pode não compreender Git, bases de dados, transações ou as particularidades de cada ferramenta. Isso não deve obrigá-lo a coordenar manualmente todos os detalhes técnicos.

O I2 Planner reconstrói a situação real, clarifica requisitos, define capacidades e dependências, propõe a estratégia e audita o terminal de uma campanha. Antes de autorizar uma alteração material deve fazer uma reconstrução completa e uma passagem adversarial independente. Esse esforço estratégico não tem de ser repetido como uma cerimónia a cada bug normal de implementação.

O I3 Controller recebe um charter autorizado e assume a execução contínua: programação, testes, integração, refatorações dentro dos limites, recuperação de falhas, correções, revisão independente e novos candidatos. Um erro de regressão não cria uma nova campanha por si só. Quando um defeito de uma mesma classe regressa repetidamente, o Controller deve investigar e endurecer o subsistema por inteiro.

O I4 é um papel opcional de execução ou revisão especializada. Não significa que sejam obrigatórios quatro modelos diferentes nem quatro conversas abertas. O I1 MASTER OWNER pode ser útil quando existe um portefólio de produtos, vários workstreams que precisam de integração ou conflitos de ownership relevantes. Para uma aplicação pessoal simples, uma organização mais pequena pode cumprir o mesmo contrato com menor custo.

Pergunta	Autoridade habitual
«Que problema vamos resolver e para quem?»	CLIENT, com discovery conduzido pelo Planner.
«Que capacidade deve ser entregue primeiro?»	Planner, respeitando prioridades confirmadas pelo CLIENT.
«Qual o índice ou transação de DB apropriados?»	Engenharia no charter, sob responsabilidade de I3.
«A funcionalidade corresponde às regras já acordadas?»	Testes e revisão técnica; auditoria estratégica do Planner.
«A aparência é boa para os utilizadores previstos?»	Gate humano/perceptual competente, incluindo o CLIENT se apropriado.
«Podemos pagar, publicar ou enviar dados para fora?»	Autoridade explícita do CLIENT ou do responsável legal aplicável.

3. Informação que deve permanecer acessível

Uma pessoa nova deve conseguir reconstruir o produto sem exigir que o CLIENT releia centenas de mensagens. Para isso é importante manter em superfícies canónicas distintas a intenção aprovada, o estado mutável atual, a autorização de uma campanha e a evidência produzida pelo runtime. Misturar tudo num único relatório longo cria contradições e dificulta saber qual dos documentos tem autoridade.

O repositório guarda código e decisões versionadas; a DB ou outro armazenamento canónico pode conter dados vivos; o Product State indica o que está atual e porquê; os

artefactos e manifests documentam resultados; logs e journals ajudam a compreender transações interrompidas. Uma mensagem recente de um agente não é suficiente para substituir uma destas fontes, mesmo quando parece convincente.

Uma situação frequente ilustra o problema. Um revisor aprovou o candidato A. Mais tarde alguém alterou o código e gerou B sem repetir a revisão. O nome da branch continua igual, mas a aprovação de A não valida B. A identidade exata da fonte, configuração, DB e artefactos evita aceitar por engano evidência de uma versão anterior.

Outro caso envolve SQLite em modo WAL. O ficheiro principal da base de dados pode não representar todo o estado atual enquanto existirem transações no write-ahead log. Uma referência ao hash do ficheiro isolado pode ser insuficiente. O sistema deve preservar um testemunho consistente da DB ou declarar incerteza concreta, sem inventar uma prova de currentness.

4. Como utilizar este manual

Estes oito capítulos formam a edição extensa. Podem ser lidos em sequência na primeira utilização, mas também funcionam como referência. O capítulo 01 trata da instalação; o 02 explica discovery e adoção; o 03 aborda sessões de trabalho; o 04 ensina a solicitar mudanças; o 05 apresenta erros e recuperação; o 06 cobre aceitação; o 07 percorre um clube fictício desde a ideia até às provas necessárias.

Se pretende experimentar o framework em poucos minutos, a miniguia em português é mais direta. É intencionalmente curta e não substitui este livro. A FAQ em português responde a questões frequentes; a documentação técnica do repositório serve de fonte precisa sobre versões e contratos.

Não é necessário memorizar cada nome de ficheiro interno. É mais importante saber que perguntas fazer, quem pode decidir, onde procurar a referência atual e quando bloquear uma alteração que depende de evidência desconhecida. O capítulo de instalação fornece comandos de exemplo, mas as opções efetivamente disponíveis devem ser verificadas na versão do script utilizado.

Para estudar, escolha um produto real ou fictício e escreva primeiro o objetivo em termos observáveis. Evite começar por desenhar uma frota de agentes ou por distribuir tarefas de código sem uma compreensão do problema. Depois compare os resultados físicos com as decisões preservadas. A experiência torna-se mais clara quando se associa cada conceito a uma ação concreta.

5. Preparação das decisões que só o CLIENT pode tomar

Antes de instalar, o CLIENT pode preparar uma descrição simples: quem utiliza o produto, que tarefa pretende executar, qual o primeiro resultado demonstrável, quais são os limites de custo e prazo, que dados são sensíveis e que ações exigem autorização. Nem tudo precisa estar decidido. A documentação deve separar requisitos confirmados de perguntas abertas, sem transformar hipóteses técnicas em vontade do utilizador.

Por exemplo, «quero gerir associados» ainda não determina se os associados pagam quotas, se há dois tipos de acesso, se uma baixa elimina o histórico ou se o sistema funciona sem internet. O Planner pergunta porque essas respostas mudam o comportamento do produto. Já a escolha entre duas estruturas de classes equivalentes costuma pertencer à engenharia e não requer uma votação do CLIENT.

É razoável pedir uma primeira demonstração pequena, como cadastrar uma pessoa fictícia e encontrá-la depois de reiniciar a aplicação. O resultado pode ser medido. «Ter uma arquitetura profissional» isoladamente é difícil de aceitar e abre espaço para produzir muitos documentos sem entregar uma capacidade utilizável.

Os requisitos mudam com aprendizagem real. A arquitetura permite registar supersessões e manter evidência útil, não congelar o primeiro pensamento para sempre. Quando surge uma decisão material nova, o Planner atualiza o contrato apropriado. Quando surge um bug de uma regra já confirmada, I3 corrige no charter existente.

6. Alcance, limites e versões

A release pública v1.4.0 foi distribuída com um ZIP portátil, checksum e manifest. Para instalar de forma reproduzível, utilize o release exato e confira o hash do ficheiro descarregado. A branch `main` pode incluir documentação e melhorias posteriores; o que aparece em `main` não passa automaticamente a fazer parte dos bytes da release anterior.

A v1.4 verificou mecanismos portáteis do framework, incluindo o scaffold, perfis, reconstrução de contexto, recuperação e testes de contratos em ambientes Linux/Windows. Isso não prova que qualquer aplicação de terceiros, integração nativa com um host específico ou serviço de IA funcione. Uma funcionalidade externa necessita dos próprios testes, permissões e gates.

O projeto é neutro quanto a modelos: não obriga à utilização de um LLM, não instala credenciais de um fornecedor e não confere permissões de cobrança ou publicação. Pode ser utilizado para coordenar trabalhos executados por agentes ou por programadores humanos. O papel da ferramenta é preservar decisões e evidências, não assegurar que uma determinada tecnologia produza software correto sem verificação.

Os perfis `SMALL`, `STATEFUL`, `ARTIFACT_HEAVY`, `MULTI_WORKSTREAM`, `MULTI_PRODUCT` e `HIGH_CONSEQUENCE` ativam famílias proporcionais de controlos. Nenhum perfil, por si só, certifica segurança, qualidade de UX ou conformidade legal. A seleção depende dos riscos concretos; projetos pequenos não devem herdar toda a complexidade de sistemas maiores sem justificação.

Também importa distinguir instalações novas de projetos já governados. O scaffold genérico não é um sistema universal de migração do Product State de uma v1.2/v1.3 para v1.4. Se há um controlador canónico, dados e campanhas ativos, começa-se com o preflight somente leitura e decide-se como reconciliar a autoridade, sem sobrescrever por conveniência.

7. Um percurso de ponta a ponta

Imagine o desenvolvimento de uma pequena biblioteca de reservas. O CLIENT define que uma pessoa autorizada deve criar reservas e verificar disponibilidade. O Planner pergunta por horários, conflitos, cancelamentos, permissões e prazo. As respostas confirmadas tornam-se requisitos de produto, com um baseline de fontes e ambiente.

Com o projeto adotado, o Planner autoriza um charter para a primeira capacidade. I3 implementa, testa, corrige falhas, integra um candidato exato e organiza a revisão apropriada. A prova física não é apenas um teste de função: alguém cria a

reserva, consulta-a depois, verifica conflitos e reinicia o programa se persistência for necessária.

O Planner audita o terminal contra o comportamento confirmado. O CLIENT verifica as questões reservadas de produto e experiência. Se um teste revelar que reservas duplicadas são permitidas apesar do requisito contrário, I3 corrige dentro da mesma campanha. Se o CLIENT acrescentar uma política nova de prioridade para associados, trata-se de uma decisão de produto e avalia-se o impacto estratégico.

Depois da aceitação, o estado canónico aponta para o candidato e evidências corretos. Um novo agente pode continuar sem recomeçar o discovery nem depender da memória de quem implementou a primeira versão. Essa é a diferença entre um chat de programação com bons resultados pontuais e um processo de desenvolvimento durável.

8. Decisões frequentes e responsabilidades

Posso usar só um chat e o repositório? Sim, se conseguir respeitar os limites e manter estado atual verificável. O contrato não exige quantidade fixa de processos. Uma pessoa ou ferramenta pode desempenhar várias operações técnicas sem inventar autoridades conflituosas.

É melhor que programar diretamente com Codex? Depende do trabalho e da implementação real. Para um protótipo pequeno, a conversa direta pode ser mais rápida. Para um produto longo com várias capacidades, alterações de contexto, revisão e necessidade de continuidade, a arquitetura pode oferecer vantagens. Essas vantagens são uma hipótese verificável, não uma superioridade medida que o manual possa afirmar sem comparar resultados, qualidade e custo.

O que acontece quando um agente erra? Se o defeito cabe no charter, I3 resolve e retesta. A responsabilidade técnica não volta ao CLIENT a cada erro. O Planner regressa quando há terminal estratégico ou uma mudança material do contrato.

O que significa `HOLD`? É a decisão de não avançar com efeitos dependentes de dados ou autoridade ainda não reconciliados. Não significa que toda a análise esteja proibida. Podem continuar investigações seguras, relatórios read-only e propostas que não dependam da incerteza.

E se o projeto for privado? Pode continuar privado. A camada de controlo não precisa de enviar código, nomes de projeto, conversas, credenciais nem dados pessoais ao mantenedor. Qualquer integração externa deve respeitar privacidade, licença e autorização do responsável.

9. Ficha de arranque de uma página

Antes da primeira campanha, confirme que consegue responder aos pontos seguintes:

Campo	Resposta necessária
Produto	Problema, pessoa utilizadora e primeiro resultado observável.
Repositório	Código verdadeiro, branch/ref atual, estado local e alterações pendentes.

Campo	Resposta necessária
Autoridade	CLIENT, Planner, Controller e eventuais owners de recursos partilhados.
Requisitos	Decisões confirmadas, dúvidas abertas e restrições de produto.
Ambiente	Linguagem, plataforma, dados, rede, custo e limites de permissões.
Estado	Product State e referências atuais sem contradições.
Campanha	Charter de capacidade, interfaces e falsificadores.
Evidência	Gates aplicáveis, candidato exato, revisões e testes físicos.
Continuidade	Próxima ação que um agente novo consegue recuperar sem ler o chat.

Se faltar um item material, não o preencha com uma suposição só para deixar a tabela verde. Identifique o owner que pode obter a resposta. A ausência de uma preferência do CLIENT não é falha de código; a ausência de um hash não é decisão de produto; a ausência de um ambiente de testes não é motivo para aceitar um resultado simulado como físico.

Esta ficha não cria um segundo registo `CURRENT`: deve servir de índice para as fontes canónicas já existentes. A duplicação de superfícies vivas pode gerar mais confusão do que aquela que a arquitetura pretende resolver.

10. Como avaliar se o sistema está a ajudar

Depois de algumas sessões, pergunte se o projeto consegue continuar após trocar de agente, se as decisões permanecem claras, se houve menos trabalho perdido e se a pessoa utilizadora recebeu uma capacidade demonstrável. Não conte apenas commits, documentos, testes ou tokens; esses indicadores de atividade não são o produto.

Uma comparação honesta com uma abordagem de chat direto deve observar tempo total, quantidade de intervenções do CLIENT, qualidade de engenharia, taxa de regressões, facilidade de recuperar contexto e comportamento real da aplicação. O framework pode exigir custo inicial maior. Esse custo é justificável apenas quando traz redução mensurável de risco ou melhora de continuidade.

Não se deve adicionar uma nova regra ou camada só porque um problema foi difícil. O princípio de proporcionalidade permanece válido durante todo o desenvolvimento: o trabalho técnico deve servir o produto, e o sistema de governação deve servir o trabalho técnico. A melhor evidência é uma capacidade que funciona e que qualquer novo responsável autorizado consegue compreender e manter.

A seguir: capítulo 01, instalação e configuração segura da camada de controlo v1.4.

Fontes e âmbito

- Visão geral da arquitetura
- Perfis de escala
- Limite de autoridade do CLIENT

- Ciclo de capacidades e evidências
- Primeiro uso público de v1.4
- Notas de lançamento v1.4

Este capítulo explica um método e as suas limitações. Não certifica aplicações concretas, integrações externas nem comparações de desempenho não realizadas.

01 — Instalação e configuração

ArquitecturaRepo · Manual completo de utilização · Edição v1.4 · Português

> Instalar o starter é criar uma camada de controlo inicial, **não adotar automaticamente a aplicação** nem demonstrar que um produto funciona. Os comandos deste capítulo seguem o início público v1.4. Um projeto novo começa corretamente em `HOLD / UNTRUSTED_CONTEXT` enquanto requisitos, fontes e autoridades não forem reconciliados.

Neste capítulo

1. Ambiente e contas
2. Starter, fork e aplicação: três lugares diferentes
3. Verificar caminhos antes de escrever
4. Escolher um perfil proporcional
5. Criar a camada de controlo
6. Validar sem inventar aceitação
7. A próxima ação é discovery
8. Antes de repetir uma instalação
9. Passar do scaffold à adoção real
10. Conexões, permissões e privacidade
11. Diagnóstico seguro

1. Ambiente e contas

Para experimentar a instalação portátil, são necessários Python 3.12, uma forma de trabalhar com ficheiros e Git, e acesso à fonte do ArquitecturaRepo. Python 3.12 é o interpretador exercitado pela CI da v1.4. Um editor como VS Code pode ajudar a inspecionar os ficheiros, mas não é obrigatório ter uma integração de IA ou conta num fornecedor externo.

Num terminal Windows, pode verificar o Python em PowerShell com os comandos abaixo. Utilize os caminhos adequados ao ambiente; executar um comando dentro de um editor não garante que utiliza a mesma instalação Python da consola.

```
& "python" --version
& "python" -m pip --version
& "git" --version
```

Se uma ferramenta não for reconhecida, investigue primeiro o executável, o PATH e a instalação. Não precisa alterar Product State para compensar uma dependência local em falta. Uma instalação de Python obtida de fonte confiável deve ser verificada antes de retomar o tutorial.

No Linux ou macOS, confirme que `python3` corresponde à versão pretendida e que o comando `python` existe se os exemplos o utilizarem. Nos comandos portáteis, as aspas delimitam caminhos com espaços. Para sistemas diferentes, a barra invertida dos exemplos PowerShell não deve ser confundida com o separador de diretórios do shell Unix.

É possível começar sem GitHub, utilizando uma cópia de fonte verificada localmente, mas Git e um repositório remoto facilitam a durabilidade de decisões e evidência. Antes de dar acesso a um agente, compreenda o âmbito das permissões: ler um repositório privado, editar ficheiros, aprovar um PR e publicar uma release são operações diferentes.

2. Starter, fork e aplicação: três lugares diferentes

Um erro frequente é misturar o repositório do framework com o repositório do produto. O starter contém documentação, protocolos, ferramentas e modelos. O fork é a sua cópia versionada do starter, caso pretenda acompanhar ou modificar essa arquitetura. O destino é a aplicação que quer desenvolver: por exemplo, uma API de reservas ou um sistema de associados.

A operação de scaffold parte da fonte do framework e escreve uma pasta `.agentic-sdlc/` dentro do **projeto de destino**, desde que não exista aí governação canónica incompatível. Não deve apagar o código de produto que já estava nesse diretório. A pasta do produto continua a ter os seus próprios requisitos, stack, dados, testes e permissões.

Local	Conteúdo e função
`ArquitecturaRepo/`	Fonte do framework, scripts de scaffold, testes e documentação.
Fork do starter	Cópia sob o seu controlo, com histórico e eventuais personalizações.
`meu-projeto/`	Código verdadeiro da aplicação, com `.agentic-sdlc/` quando instalado.
`.agentic-sdlc/`	Contratos e estado de coordenação do produto, não o produto em si.

Os exemplos de projeto incluídos no framework são sintéticos. Não representam a sua base de dados, não substituem os requisitos do CLIENT e não podem ser copiados como se fossem um estado de produção autorizado. Instalar uma camada de governo sem compreender o produto cria um ponto de partida, não um resultado validado.

3. Verificar caminhos antes de escrever

Antes de executar o instalador, confirme a pasta onde está o terminal, a origem e o destino. Em PowerShell, uma verificação simples permite distinguir a fonte do projeto que será alterado:

```
& "python" "tools\scaffold_project.py" --help
```

Este comando de ajuda deve ser executado na raiz do starter e é de leitura. Reveja as opções realmente suportadas pela cópia instalada. Se não existir `tools\scaffold_project.py`, provavelmente está na pasta errada ou não tem a distribuição esperada. Não tente reparar copiando ficheiros aleatórios de outra versão.

O destino deve ser explícito e específico. Por exemplo, `"..\meu-projeto"` refere-se a uma pasta adjacente, não ao diretório do sistema e não a um caminho universal. Se a pasta já contiver uma aplicação, faça inventário dos ficheiros e da governação antes

de iniciar. Se nela existir um Product State, um controlador ativo ou um esquema de dados que o framework não conhece, utilize a rota de leitura para projetos já governados.

Para uma instalação reproduzível, descarregue a release v1.4.0, que contém ZIP portátil, ficheiro de checksum e manifest. Confira o SHA-256. Pode também trabalhar com um checkout do tag. A branch `main` é móvel; não atribua ao ZIP publicado ficheiros que só foram adicionados depois.

4. Escolher um perfil proporcional

O framework dispõe de seis perfis com famílias de controlo diferentes. Não são níveis de prestígio e nenhum deles faz a aplicação funcionar automaticamente:

Perfil	Quando o considerar
`SMALL`	Aplicação pequena, com riscos modestos e sem persistência material complexa.
`STATEFUL`	Dados duráveis, esquemas, utilizadores, recuperações, backups ou runtime com estado.
`ARTIFACT_HEAVY`	Muitos ficheiros, recursos caros, lineage, licenças e políticas de utilização.
`MULTI_WORKSTREAM`	Linhas de trabalho diferentes que integram um único produto final.
`MULTI_PRODUCT`	Produtos realmente distintos com recursos ou políticas partilhadas.
`HIGH_CONSEQUENCE`	Risco material de segurança, privacidade, integridade ou efeitos difíceis de reverter.

Para uma primeira experiência sem requisitos especiais, `SMALL` costuma ser adequado. Um clube com registo de associados e quotas persistentes pode justificar `STATEFUL`. Um sistema de edição de vídeos que rastreia centenas de recursos pode necessitar de controlos de artefactos e direitos. A escolha deriva do comportamento e do risco, não de uma vontade de criar mais agentes.

O perfil não concede automaticamente permissões de rede, escrita em bases de produção ou despesas. Ainda é necessário descobrir a fonte real, definir ownership e confirmar com o CLIENT as decisões que alteram o produto. A documentação técnica dos perfis e modos de projeto explica as diferenças.

5. Criar a camada de controlo

Na raiz do starter v1.4, pode criar uma camada de controlo para um projeto novo. Em PowerShell, com caminhos e valores explicitamente delimitados:

```
& "python" "tools\scaffold_project.py" --target "..\meu-projeto" --profile "SMALL" --p
```

Para o exemplo fictício de um clube com estado persistente:

```
& "python" "tools\scaffold_project.py" --target "..\clube-app" --profile "STATEFUL" --
```

No shell Unix, utilize o script com /, por exemplo "tools/scaffold_project.py", e o destino "../meu-projeto". Adapte product-id, objetivo e pasta aos dados legítimos da aplicação. O instalador deverá recusar sobrescritas de controlo canónico e preservar ficheiros preexistentes que não lhe pertencem.

O estado inicial HOLD / UNTRUSTED_CONTEXT é o resultado esperado. Ele indica que o sistema não fabricou decisões do CLIENT, um baseline de código, um Goal em execução ou uma capacidade aceite. Escrever manualmente READY em um JSON para obter uma validação verde não altera os factos, apenas esconde a lacuna.

A opção --with-runtime, quando aplicável, acrescenta mecanismos portáteis de referência para campanhas; não autentica uma conta de Codex Desktop, não inicia um LLM e não concede acesso a provedores externos. --no-feedback-workflow evita o agendamento periódico, mas não desativa o gerador local de relatórios por si só. Consulte a configuração instalada antes de ativar fluxos de rede.

6. Validar sem inventar aceitação

Depois de criar o scaffold, abra o **diretório do projeto de destino**, não permaneça na raiz do framework. Em PowerShell, execute:

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-sdlc"
```

O primeiro instala dependências locais declaradas. O project_status.py apresenta estado inicial; discover_project.py recolhe inventário, metadados e hashes de ficheiros sem enviar o conteúdo do código ao mantenedor; reconstruct_context.py tenta recuperar a situação a partir dos registos disponíveis. A discovery deve respeitar limites de caminhos, links simbólicos e destinos fora do projeto.

Algumas verificações podem devolver HOLD enquanto não existir adoção confirmada. Isso é correto. A validação de contratos verifica relações e estruturas da camada de controlo; não demonstra que o backend da aplicação se ligue à DB, que a UI abra ou que um utilizador consiga recuperar uma reserva depois de reiniciar.

Registe o estado, os erros e a versão de código utilizada. Confirme que a aplicação anterior continua intacta. Se faltar jsonschema, instale as dependências declaradas, não edite um estado canónico para evitar a verificação. Se um comando falhar por permissões, não contorne a proteção com uma eliminação ou instalação forçada.

O resultado desta fase é **camada inicial instalada e inspecionada**. A adoção da aplicação real exige ainda discovery, requisitos aprovados, baseline, owner, currentness e autorização do charter. O sucesso de instalação deve ser relatado com esse limite explícito.

7. A próxima ação é discovery

O Planner deve começar pela aplicação que existe, pelos utilizadores previstos e pelas decisões confirmadas. Os prompts de instalação e adaptação, discovery de produto e transferência/adoção servem como referências. Não entregue imediatamente à engenharia uma lista genérica de funcionalidades baseada apenas no exemplo do manual.

Discovery identifica: problema, critérios observáveis de aceitação, código e DB

atuais, interfaces e dependências, restrições de prazo e custos, riscos de privacidade, tecnologia existente e responsabilidades. O CLIENT confirma decisões de produto; a engenharia comprova afirmações técnicas. O Planner separa hipóteses de requisitos efetivamente aceites.

Depois, prepara-se uma especificação explícita de adoção. Faz-se preflight contra as fontes atuais, promove-se o estado de modo coerente e verifica-se se uma sessão fresca consegue recuperar o mesmo contexto. Só após esse baseline pode ser autorizada uma campanha material com limites reais.

Uma boa instalação não acelera o projeto se os agentes começarem a modificar ficheiros sem saber qual o requisito. A etapa de discovery é a ponte entre um framework genérico e um produto específico. O custo inicial desta ponte deve ser proporcional à dimensão e ao risco da primeira entrega.

8. Antes de repetir uma instalação

Se `.agentic-sdlc/` já existir, pare e investigue. Pode conter Product State, campanhas, owners, journals, fontes adotadas e referências a dados que não devem ser perdidos. Não apague a pasta para repetir o exemplo. Não use uma opção de força como método de resolução de erros e não inicialize um segundo controlador sobre o primeiro.

Quando a aplicação já possui governação, use o preflight de attachment somente leitura. Essa rota pode testar referências ao código e consistência de snapshots de SQLite/WAL, mas não é um atualizador automático de qualquer v1.2 ou v1.3. Uma migração material exige uma decisão com owners e evidência apropriados.

Se um comando falha após uma interrupção, antes de repetir confirme se houve efeito parcial. Ficheiros podem ter sido criados, journals podem estar pendentes e uma operação externa pode ter ocorrido mesmo que o terminal tenha perdido a ligação. Primeiro faça readback e preserve evidência; depois o Controller escolhe uma recuperação idempotente.

Nem todo problema de instalação pertence ao Planner. Dependências, caminhos, permissões e ajustes dentro de um charter são assuntos de execução. Uma incompatibilidade de autoridade ou alteração material do contrato pode exigir reconstrução estratégica. Separar essas classes evita ciclos de planeamento desnecessários.

9. Passar do scaffold à adoção real

Um percurso adequado contém etapas distintas: inventariar o código; confirmar os requisitos com o CLIENT; estabelecer baseline de produto e baseline técnico; escolher estratégia e marcos datados; preparar a especificação de adoção; executar preflight; promover estado e ownership coerentes; validar fresh-context; e somente então autorizar uma capacidade material.

Cada etapa produz evidência que será utilizada pela seguinte. O baseline técnico não pode apontar para um commit antigo enquanto a campanha foi construída sobre outro. O estado da DB não deve ser descrito por um hash insuficiente. O charter não pode afirmar que existe uma credencial, licença ou integração que ninguém verificou.

O CLIENT não é obrigado a reunir hashes, gerir leases ou decidir entre dois scripts de recuperação. O sistema deve permitir que se concentre no comportamento desejado,

no orçamento e na aceitação humana. A engenharia deve tornar os limites e as falhas compreensíveis, sem transferir a responsabilidade técnica para quem pediu o produto.

Se o destino for uma aplicação já em desenvolvimento, preserva-se código e evidência úteis. Adotar a arquitetura não é uma autorização para reescrever tudo. O trabalho começa por compreender o que já funciona, o que falta e onde existe risco material que as ferramentas conseguem realmente reduzir.

10. Conexões, permissões e privacidade

Os conectores para GitHub, ferramentas de programação, bases de dados ou serviços de IA dependem de permissões configuradas no ambiente. O instalador não traz tokens nem autenticadores e não consegue garantir a sessão nativa de um fornecedor. Se a integração falhar, diagnostique o âmbito do acesso e o erro sem pedir ao CLIENT que exponha credenciais numa conversa.

O gerador local de feedback de arquitetura utiliza metadados restritos. Na instalação padrão, pode existir um workflow mensal que guarda relatórios nos artefactos do próprio GitHub Actions do projeto. O pacote público v1.4 não inclui um endereço de coleta externo. `--no-feedback-workflow` desativa o agendamento; para desligar inteiramente a geração, configure `enabled=false` no ficheiro instalado de feedback. A política de privacidade detalha os limites.

Nunca copie código privado, identificadores de pessoas, bases completas, logs com tokens ou conversas para um issue público. Um bug pode ser relatado com ambiente, comando, resultado esperado, erro sanitizado e exemplo fictício. Antes de ativar efeitos de rede, despesas, publicação ou envio de dados, obtenha a autorização específica adequada.

Licenças também importam: o framework ter licença MIT não transforma automaticamente imagens, fontes, bases, serviços ou documentos de terceiros em recursos livres. O responsável pela aplicação deve avaliar direitos de acordo com o uso efetivo e com a evidência disponível, sem impor controles irrelevantes nem ignorar restrições legais reais.

11. Diagnóstico seguro

Se o Python não iniciar, confirme executável e versão. Se faltar uma dependência, instale a lista declarada. Se um caminho não existir, compare diretório atual, origem e destino. Se aparecer `EXISTING_CONTROL_PRESERVED`, não tente sobrescrever: siga a rota de attachment. Se o Project State estiver inconsistente, investigue as referências atuais antes de qualquer escrita.

Para problemas de Git, preserve worktree e branches; para SQLite, preserve snapshots consistentes e WAL; para operações interrompidas, preserve journal e identidade da tentativa. O capítulo 05 aprofunda a recuperação. A regra comum é diagnosticar a causa e demonstrar o gate afetado, não apenas fazer desaparecer a mensagem.

Um bom relatório de instalação contém versão exata do starter, pasta de destino, perfil, preservação dos ficheiros existentes, comandos executados, resultados de validação e limites abertos. Nunca deve concluir que a aplicação foi desenvolvida ou aceite apenas porque o scaffold iniciou corretamente.

A seguir: capítulo 02, o Planner, o discovery e a adoção de um projeto real.

Fontes e âmbito

- Primeiro uso e suporte público v1.4
- Scaffold e perfis
- Transferência e adoção
- Attachment de projeto com governação existente
- Privacidade e feedback
- Instalador de referência

Os exemplos são ilustrativos e não alteram as condições do ZIP v1.4.0 já publicado. Instalar o starter não concede autoridade para executar campanhas, utilizar fornecedores nem modificar uma aplicação já governada.

02 — O Planner e a adoção do projeto

> A presença de `.agentic-sdlc/` não prova que a aplicação tenha sido compreendida. O Planner tem de relacionar o framework com requisitos confirmados, código real, dados atuais, ambiente, decisões técnicas e autoridade. O objetivo da adoção é passar de `UNTRUSTED_CONTEXT` para um estado reconstruível **sem inventar factos nem criar um segundo sistema canónico**.

Neste capítulo

1. Uma conversa sobre o projeto, não sobre a ferramenta
2. Preparar um espaço de trabalho
3. Discovery: fazer as perguntas corretas
4. Baseline de produto e plano de entrega
5. Adoção: relacionar starter e aplicação
6. O exame decisivo de um contexto fresco
7. Primeira campanha autorizada
8. Exemplo de sessão inicial com o Planner
9. Um baseline confirmado pelo CLIENT
10. Exemplo de primeira campanha

1. Uma conversa sobre o projeto, não sobre a ferramenta

O Planner começa pelo produto final. Uma pessoa pode dizer «quero um sistema de reservas para o clube» ou «preciso de uma API para organizar dados». Antes de falar em agentes, workflows ou ficheiros de estado, é necessário compreender quem utilizará o sistema, que problema existe hoje e como se poderá observar uma melhoria real.

Uma descrição útil distingue o trabalho do utilizador do trabalho técnico. O CLIENT pode decidir que uma reserva cancelada deve libertar um lugar imediatamente, mas não precisa escolher a implementação de uma transação. A engenharia pode propor uma base relacional, mas não deve decidir sem consulta que os dados antigos podem ser eliminados. O Planner mantém essas responsabilidades separadas.

Perguntas centrais são: quem usará o produto, em que ambiente, que dados são necessários, quais as funcionalidades realmente prioritárias, que integrações já existem, qual o limite de custo e prazo, e que situações negativas são perigosas. As respostas têm de ser confirmadas antes de se tornarem requisitos vigentes; hipóteses provisórias podem ser registadas como perguntas abertas.

Não é preciso transformar o primeiro contacto numa entrevista interminável. Para um protótipo simples, poucas perguntas podem bastar para começar uma capacidade pequena. Para um sistema com dinheiro, dados pessoais ou dependência de produção, o nível de investigação deve aumentar. A proporcionalidade é uma decisão de risco, não de formalismo.

2. Preparar um espaço de trabalho

O Planner deve saber qual é a fonte do código e qual é o projeto de destino. Antes de editar, inspecciona a estrutura de pastas, o estado de Git, a branch, alterações locais, configurações relevantes, testes existentes, DBs, scripts de arranque e a camada de controlo, se existir. Um checkout parecido pode conter fontes diferentes; a identidade não é estabelecida pelo nome da pasta.

A análise inicial deve ser principalmente de leitura. O objetivo é descobrir o que já existe e o que funciona. Para um projeto antigo com governação própria, o preflight de attachment existente impede que uma instalação genérica crie uma segunda autoridade. Para um projeto novo sem governação, o scaffold inicial pode ser apropriado, mas continua em HOLD até concluir a adoção.

Uma ferramenta de descoberta pode enumerar ficheiros e hashes. Essa informação ajuda, mas não permite inferir automaticamente regras de negócio. Ler uma tabela chamada `payments` não comprova quais os tipos de pagamento, quem pode rever um registo ou se um lançamento pode ser anulado. O Planner tem de verificar com o CLIENT o significado funcional e com a engenharia o comportamento efetivo.

É importante separar material público do framework de artefactos privados do projeto. Relatórios com dados reais não devem ser colocados num repositório público para facilitar a continuidade. Pode-se preservar a identidade de um artefacto por referência e hash sem copiar o seu conteúdo para toda a equipa.

3. Discovery: fazer as perguntas corretas

Boas perguntas produzem decisões que alteram o produto. Para um clube fictício, por exemplo: pode existir um associado com mensalidades em atraso? Uma inscrição depende de estar em dia? Como se corrige um pagamento registado por engano? O que acontece a um histórico quando se desativa uma conta? É possível operar sem ligação à internet?

Uma pergunta trivial, cuja resposta já está documentada na fonte, desperdiça a atenção do CLIENT. O Planner deve primeiro ler o que existe e perguntar apenas pelos pontos de produto ainda indefinidos ou contraditórios. Uma pessoa que pede uma aplicação não deve ter de responder a dezenas de perguntas sobre nomes de classes ou frameworks que a engenharia consegue resolver.

Tipo de pergunta	Exemplo	Consequência
Necessidade	«Que tarefa demora mais hoje?»	Define a primeira capacidade de valor.
Regra de negócio	«Uma pessoa pode reservar com uma quota em atraso?»	Afeta elegibilidade e mensagens.
Dados	«É obrigatório conservar o histórico de pagamentos?»	Afeta persistência e recuperação.
Utilização	«Há dois operadores em simultâneo?»	Afeta concorrência e experiência.
Segurança	«Quem pode ver contactos pessoais?»	Afeta autorização e exposição.
Limites	«Qual é a data útil da primeira versão?»	Afeta escopo e critical path.

O Planner confirma as respostas e regista-as como requisitos com fonte e data. Se o CLIENT responder «ainda não sei», mantém a questão aberta e evita iniciar trabalhos que dependam de uma hipótese não autorizada. Pode, entretanto, avançar naquilo que é independente da decisão pendente.

O prompt de discovery de produto e a documentação sobre requisitos e entrega fornecem referências. São meios de investigação, não substitutos da responsabilidade do CLIENT por escolhas relevantes.

4. Baseline de produto e plano de entrega

Uma vez confirmada a necessidade, o Planner regista uma linha de base: objetivo, público, comportamentos, critérios de aceitação, exclusões, restrições e decisões abertas. Separadamente, engenharia reconstrói uma linha de base técnica: linguagem, bibliotecas, banco, interfaces, ambiente de execução, proteção de dados, testes e eventuais serviços externos.

Estas duas linhas de base não são intercambiáveis. Uma escolha técnica não prova que o CLIENT desejou o comportamento; uma especificação de produto não prova que o código atual a realiza. O vínculo entre ambas deve apontar para fonte concreta e estado atual verificável.

O plano de entrega divide o resultado em capacidades demonstráveis e dependências. Para um clube: identidade de associados, mensalidades, atividades, inscrições. Se uma capacidade depende da anterior, o Planner deve evitar autorizar várias campanhas incompatíveis sobre a mesma interface. O critical path deve ser revisto quando mudam datas, custos ou requisitos.

Marcos como «cadastro utilizável» são melhores que «100 commits de backend». O primeiro permite observar uma tarefa; o segundo mede atividade. Uma previsão tem de explicitar pressupostos e riscos, não transformar uma data pretendida em promessa de entrega sem evidência de capacidade.

O resultado pode ser compacto. A qualidade vem de decisões verificáveis, owners e interfaces claras. Não há benefício em manter vários ficheiros que duplicam os mesmos requisitos atuais e se contradizem após cada mudança. Use a superfície canónica prevista pelo projeto.

5. Adoção: relacionar starter e aplicação

A adoção transforma um controlo genérico num sistema relacionado com a aplicação real. Não consiste em copiar os ficheiros do exemplo nem em atribuir valores arbitrários a schemas. O Planner e a engenharia identificam Product State, donos das fontes, referências Git, dados e testigos atuais, mapeiam interfaces e estabelecem uma rota de recuperação.

O contrato de transferência e adoção descreve formatos e verificações. A especificação concreta tem de ser validada antes de promover o estado. O processo não deve fazer mutações materiais enquanto existirem contradições de autoridade, snapshots de DB insuficientes ou dependências que não foram reconfirmadas.

Um projeto com código mas sem governação pode receber a camada de controlo de forma não destrutiva. Um projeto que já possui outro Product State requer uma rota de attachment de leitura, migração cuidadosamente autorizada ou convivência explicitamente delimitada. A palavra «instalação» não é licença para substituir a fonte da verdade existente.

Os dados persistentes merecem especial cuidado. Um snapshot válido deve representar estado transacionalmente consistente quando a aplicação utiliza SQLite com WAL ou outro mecanismo equivalente. Um hash do ficheiro principal isolado não prova que as últimas escritas estejam representadas. Se faltar um testemunho relevante, o estado dependente deve ficar em HOLD.

O mesmo vale para os artefactos. Não basta apontar para `final.pdf` ou `demo.mp4` se o conteúdo mudou. Versão, hash, fonte, contexto de geração e rights devem ser rastreáveis quando afetam aceitação ou reutilização. O custo de guardar evidência deve ser proporcional: não duplique gigabytes de ficheiros só para criar uma cópia para cada agente.

6. O exame decisivo de um contexto fresco

Uma adoção ainda não é confiável se só o agente que a escreveu consegue interpretá-la. O teste de contexto fresco pede que outra sessão, sem memória da conversa anterior, descubra o projeto, identifique o Product State, distinga requisitos confirmados de perguntas abertas e indique a próxima ação permitida.

O comando `reconstruct_context.py` instalado oferece uma verificação portátil desse mecanismo. Pode detetar relações incoerentes ou fontes não reconciliadas, mas não substitui uma leitura competente da aplicação. A reconstrução deve ser conferida com evidência real: ref atual, owners, campanha, plano, estado da DB, artefactos e eventuais journals pendentes.

Se um novo agente identifica dois charters como atuais para a mesma capacidade ou encontra um relatório que aponta para código antigo, deve preservar a contradição e investigar. Não escolhe o estado mais recente por data de ficheiro nem assume que o último chat tinha razão. O sistema falha de forma segura quando não consegue demonstrar currentness.

Um resultado `HOLD` pode ser correto. Deve dizer qual dado falta, porque importa e quem pode obtê-lo. Uma mensagem vaga «não consigo continuar» é menos útil que «a DB usa WAL e não há snapshot consistente correspondente ao ref autorizado; leituras seguras continuam, mas escrita material está suspensa».

7. Primeira campanha autorizada

Depois de adoção coerente, o Planner prepara a primeira campanha por capacidade. O charter delimita o que será entregue, porque importa, que interfaces serão afetadas, responsabilidades, dependências, critérios de aceitação, riscos e falsificadores. Não prescreve cada ficheiro e microtarefa que I3 deve modificar.

Uma passagem estratégica completa examina o produto final, currentness, DB, recuperação, segurança, custos, concorrência, licenças, revisão humana e interações com outras capacidades. Uma segunda passagem adversarial procura contradições ou pressupostos frágeis. Uma terceira é excecional, justificada por risco sistémico ou evidência nova material, não por qualquer bug encontrado num teste.

Depois da autorização, I3 tem espaço operacional para implementar e corrigir. O Planner não deve interromper para reformular a arquitetura a cada falha de CI, refactor interno ou necessidade de retry. Se a revisão terminal identificar problemas dentro do charter, devolve falsificadores consolidados à mesma campanha.

É aqui que a autonomia técnica se torna concreta: o CLIENT aprova a direção e os limites do produto; o Planner governa a estratégia; o Controller trata das dificuldades

de engenharia até existir uma entrega verificável. O trabalho pode usar um ou vários especialistas, mas a autoridade e a evidência não se perdem entre eles.

8. Exemplo de sessão inicial com o Planner

Uma pessoa diz: «Quero uma aplicação local para controlar inscrições num clube. Hoje tenho uma planilha e preciso de algo que funcione daqui a um mês». O Planner não deve responder com uma escolha aleatória de cloud, microserviços e agentes. Primeiro investiga se existem dados reais, quem utiliza a ferramenta e qual a menor tarefa que precisa de melhorar.

Uma resposta organizada começa por esclarecer o objetivo, confirmar se há mais de um operador e perguntar quais regras determinam inscrição e cancelamento. Depois identifica a pasta de código existente, se existe, e o ambiente onde a aplicação terá de executar. Se o prazo é curto, propõe uma primeira capacidade que possa ser testada e uma lista explícita de exclusões.

O Planner regista decisões e prepara o baseline. Se a aplicação já tiver código com autenticação e uma base de dados, não reimplementa tudo só para encaixar o starter. Se o repositório estiver vazio, escolhe uma tecnologia proporcional e explica impactos significativos de manutenção ou instalação. O produto e os riscos determinam o stack, não o inverso.

No fim da sessão o CLIENT deve saber o que foi confirmado, o que ficou em aberto, qual é o primeiro resultado a ver e que decisão ainda precisa tomar. Não precisa compreender a sintaxe de todos os schemas de governança para conseguir aprovar um objetivo.

9. Um baseline confirmado pelo CLIENT

Um resumo de uma página pode dizer: «O sistema permite a um administrador registar associados, consultar o histórico e gerir mensalidades em computador local. Não processa pagamentos online. O histórico deve ser mantido após a desativação. A primeira demo deve realizar alta, pesquisa, edição e readback persistente». Se estas frases forem confirmadas pelo CLIENT, podem virar requisitos explícitos.

A engenharia acrescenta informação verificável: código e ref adotados, versão do interpretador, biblioteca de persistência, formato dos backups, eventuais riscos de dados pessoais e comandos de teste. As preferências ainda desconhecidas permanecem identificadas como pendentes; não se inventam aprovações ou prazos de terceiros.

Esse baseline permite rejeitar uma funcionalidade que não responde ao pedido. Se alguém criar um painel bonito mas não conseguir gravar um associado, o resultado visual é insuficiente. Se alguém adicionar pagamentos online apesar da exclusão, ultrapassou o contrato. Os critérios de aceitação devem observar a experiência de uso, os dados e o comportamento real.

Um requisito pode mudar após feedback. Nesse caso regista-se a supersessão e avaliam-se downstream e compatibilidade com o charter. Não se mantém simultaneamente uma regra antiga e uma regra nova como verdade canónica para o mesmo assunto.

10. Exemplo de primeira campanha

Uma primeira campanha pode ter como objetivo «Gestão de associados com persistência verificável». O resultado esperado inclui criar, encontrar, editar e desativar um associado fictício, mantendo histórico relevante. As interfaces afetam formulário, API ou camada de aplicação, operações de DB e pesquisa. O charter define limites de dados e segurança e deixa o desenho interno ao Controller.

Os gates poderiam envolver testes unitários de validação, integração de criação e pesquisa, readback após reinício, casos negativos de duplicados, segurança de permissões, prova física da interface e revisão independente. O CLIENT participa onde comportamento e experiência exigem julgamento humano. Nenhum gate substitui outro que tenha relevância para a funcionalidade.

Se uma desativação apagar pagamentos anteriores apesar do requisito, I3 corrige no mesmo charter. Se o CLIENT pedir uma função de exportação fiscal que não estava prevista, o Planner avalia a nova necessidade, os dados e as autorizações. Saber distinguir estas rotas evita burocracia excessiva e alterações não consentidas.

Ao concluir, o Controller apresenta identidade exata do candidato, testes realmente executados, evidência dos passos, reviewer, limitações e resultado. O Planner decide aceitar ou devolver falsificadores. O novo Product State deve refletir a decisão competente, e a próxima campanha deve depender da capacidade que foi de facto aceite.

A seguir: capítulo 03, operação diária, controle de execução e continuidade.

Fontes e âmbito

- Discovery de produto
- Requisitos e entrega
- Transferência e adoção
- Attachment de governação existente
- Padrão de charter
- Limites do CLIENT

Um baseline só é atual quando vinculado a decisões, fontes e estado verdadeiros. Os exemplos de clube deste manual não servem como aprovação de um produto real.

03 — Operação diária

> O trabalho quotidiano começa pelo estado **CURRENT**, pela fonte e pela evidência, não apenas pela última mensagem de um chat. A engenharia tem autonomia dentro de um charter aprovado; os limites de autoridade, privacidade, custo e revisão continuam a aplicar-se a cada operação material.

Neste capítulo

1. Preparar uma sessão sem recomeçar do zero
2. Converter uma intenção numa capacidade verificável
3. O trabalho contínuo do Controller
4. Autorizações e efeitos externos
5. Como ler criticamente um relatório de execução
6. Git e continuidade entre contextos
7. Gestão de recursos, artefactos e espaço
8. Coordenação, concorrência e intervenções do CLIENT
9. Revisão independente e candidatos exatos
10. Uma rotina curta e reutilizável

1. Preparar uma sessão sem recomeçar do zero

Num projeto de várias semanas, uma nova conversa não significa um projeto novo. O primeiro passo é reconstruir a intenção aprovada e o estado atual a partir do repositório, Product State, DB, filas e artefactos canónicos. A memória do agente e um resumo anterior podem ajudar a localizar informação, mas não podem prevalecer sobre evidência física viva que os contradiga.

O Planner ou o Controller identifica a branch correta, commit ou versão da fonte, alterações locais ainda não integradas, Goal e charter atuais, owners, checkpoints, testigos de DB e gates pendentes. Se os nomes coincidirem mas os hashes não, a diferença deve ser investigada. Se o estado for desconhecido, descreve-se a incerteza concreta e mantém-se HOLD sobre as alterações materiais que dependem dela.

Uma sessão de engenharia pode começar com uma ação simples: verificar se o último terminal pertence realmente ao candidato atual. Quando o reviewer avaliou A e o código mudou para B, a aprovação antiga não serve automaticamente para fechar a campanha. O trabalho seguinte é determinar quais os controles que precisam ser repetidos, preservando a evidência estável que continua aplicável.

O procedimento de bootstrap e o protocolo de reconstrução ajudam a orientar a recuperação. O operador não deve declarar `READY` com base numa frase confiante; tem de apresentar fontes que outra pessoa possa localizar e verificar.

2. Converter uma intenção numa capacidade verificável

O CLIENT pode pedir «quero ver as mensalidades em atraso numa lista clara». O Planner confirma o significado de atraso, o período considerado, quem pode consultar a informação e como se observa o resultado. Também verifica se já existe uma capacidade anterior de pagamentos da qual a nova lista dependerá.

Um charter de capacidade descreve objetivo, limites, interfaces, dependências, owners, critérios de aceitação e falsificadores. Não precisa decidir antecipadamente todos os nomes de ficheiros e passos de execução. A divisão em tarefas, programação e tratamento de falhas competem a I3 depois da autorização.

É útil explicitar exclusões. Se a funcionalidade é uma consulta interna, ela não implica criar automaticamente um sistema de pagamentos online, notificações ou exportações fiscais. São mudanças de produto diferentes que exigem decisão própria. A engenharia não deve acrescentá-las porque a biblioteca técnica permitiria fazê-lo.

Quando um pedido é ambíguo, o Planner separa a parte segura da parte que exige nova decisão. Pode investigar estrutura de dados ou elaborar protótipos não materiais, mas não deve alterar dados reais com uma regra suposta. Uma boa pergunta ao CLIENT é curta, ligada a uma consequência e feita apenas quando a resposta não pode ser obtida nas fontes já autorizadas.

3. O trabalho contínuo do Controller

O I3 Controller é o responsável operacional por uma campanha autorizada. Pode organizar execuções técnicas, implementar, depurar, testar, recuperar crashes, refatorar dentro do scope, corrigir avaliadores, coordenar revisão independente e integrar candidatos sucessivos. Não regressa ao Planner por cada erro normal encontrado no percurso.

Uma falha de teste de integração pode exigir mudar duas classes, atualizar uma migração e melhorar o teste. Se tudo continua a satisfazer o mesmo charter, essa é execução interna. Uma revisão que deteta uma regressão deve devolver evidência precisa a I3; não criar uma nova campanha apenas para corrigir a regressão.

Quando a mesma classe de defeito regressa várias vezes, o Controller deve investigar a causa no subsistema, em vez de explorar um falso terminal após outro. Uma correção sólida pode envolver testes de regressão, revisão dos limites de transação, inspeção de contratos e uma avaliação independente contra o novo candidato. O número de rondas não é o objetivo; o desbloqueio da capacidade é.

O Controller pode recorrer a especialistas quando o problema é separável e a autoridade está definida. Quatro papéis não precisam virar quatro agentes sempre ativos. Concurrency só se justifica quando worktrees, DBs, ficheiros e recursos afetados permitem trabalho sem interferência material.

4. Autorizações e efeitos externos

Nem toda operação técnica necessita de aprovação humana individual. Ler código, executar testes locais, fazer commits dentro de um charter e recuperar uma falha conhecida podem estar no mandato de I3. Já publicar, enviar mensagens, gastar dinheiro, apagar dados importantes, enviar código privado a um serviço externo ou modificar uma base de produção pode exigir autorização específica.

Ação	Tratamento razoável
Ler código e realizar verificações autorizadas.	Operação de engenharia normal.
Corrigir implementação dentro do charter.	Responsabilidade de I3, com ownership e revisão adequados.
Instalar dependências novas.	Confirmar procedência, necessidade, segurança e permissões.
Publicar, realizar pagamento ou apagar dados relevantes.	Validar autoridade e preflight específico antes do efeito.
Copiar credenciais ou dados privados para uma conversa.	Não o fazer; utilizar mecanismos de segredo aprovados.
Encontrar contradições de owner ou DB.	HOLD dependente e reconciliação baseada em evidência.

Um CLIENT não deveria decidir entre dois índices SQL semelhantes, gerir conflitos de merge técnicos ou escolher a semântica de idempotência de uma operação já especificada. Deve, porém, decidir se o produto aceita uma determinada política de retenção, a experiência esperada ou um efeito de custo que ultrapassa o limite autorizado.

O starter inclui ferramentas de referência como `campaignctl.py`, mas a existência desses scripts não demonstra que um host externo tenha iniciado uma operação nativa de Codex ou outro fornecedor. Antes de declarar uma execução real, é preciso verificar o efeito no sistema alvo e não somente a intenção do controlador.

5. Como ler criticamente um relatório de execução

Um relatório útil indica o que foi tentado, o candidato exato, ambiente, comandos e dados de teste, resultado físico, mudanças feitas, resultados de gates, revisão independente e limitações. Também aponta para a próxima ação e para o owner responsável. Sem essas referências, termos como `PASS`, `READY` e `DONE` podem ocultar diferenças importantes.

Uma prova unitária confirma lógica isolada; um teste de integração verifica componentes juntos; uma prova de DB/currentness aborda dados e referências atuais; uma avaliação semântica verifica a regra de negócio; uma prova física executa o percurso real; um gate perceptual requer julgamento humano quando aplicável; um gate de release verifica a distribuição exata.

Uma prova de que um botão é renderizado não mostra que funciona. Um backend verde não demonstra que o utilizador consegue encontrar a função. Uma fixture sintética não é uma aplicação em produção. A pergunta útil é: «Qual gate foi exercitado, sobre que candidato e com que evidência?». Essa pergunta evita aceitar capacidades apenas pela confiança do agente.

O terminal deve distinguir o que passou, o que está aberto e o que precisa de decisão de produto. Não deve obrigar o CLIENT a ler logs enormes, mas também não pode esconder um risco material. Um resumo curto e preciso com links canónicos é mais útil que uma narrativa heroica de horas de execução.

6. Git e continuidade entre contextos

Git preserva mudanças versionadas, mas um commit não identifica automaticamente o ator, a campanha ou o owner da decisão. Também não prova que a DB esteja coerente, que a revisão inspecionou o mesmo candidato ou que um artefacto publicado corresponda ao código mais recente. Cada relação necessária deve ser documentada nas superfícies adequadas.

Quando existem alterações locais, preserve-as antes de sincronizar. Um `git pull` cego sobre um worktree com ficheiros importantes pode gerar conflitos; uma limpeza destrutiva pode apagar evidência. Compare base, ref remoto, alterações do utilizador e ownership antes de integrar. Os nomes das branches ajudam a navegar, mas os hashes e os registos de autoridade fazem a diferença.

Um commit de checkpoint pode ser valioso para recuperação, sem representar aceitação. Do mesmo modo, cinco commits pequenos não equivalem a cinco capacidades entregues. Um relatório de progresso deveria falar de resultados observados e riscos reduzidos, não de quantidade de movimento no Git.

As ferramentas de delta do repositório permitem inspeção incremental. Se um agente novo não conseguir identificar a fonte atual com segurança, deve investigar antes de modificar. Uma conversa passada não autoriza adivinhar o estado só porque a última mensagem dizia que «estava tudo pronto».

7. Gestão de recursos, artefactos e espaço

O espaço em disco é um recurso do projeto. Não crie clones completos repetidos para cada agente se existe um checkout canónico seguro. Quando a concorrência exigir isolamento, prefira worktrees temporárias que partilham o banco de objetos Git e remova-as quando deixam de ser necessárias, preservando os registos relevantes.

Renders, builds, áudio temporário, screenshots e caches podem consumir muito espaço. Antes de gerar um artefacto pesado, verifique se já existe uma versão canónica com identidade e lineage adequados. Evite várias cópias do mesmo vídeo ou PDF com nomes diferentes apenas para cada reviewer. A evidência pode apontar para o mesmo artefacto por hash e caminho autorizado.

O `tools/workspace_gc.py` e os protocolos de inventário e limpeza de artefactos tratam descarte baseado em ownership. Uma execução de limpeza deve começar com inventário e dry-run. Não se removem dados humanos desconhecidos, source code, artefactos aceites ou conteúdo canónico só porque parecem antigos.

O custo também inclui tempo de compute, APIs e limites de fornecedores. Uma operação pode ser pequena em código e cara em geração de mídia; outra pode exigir testes demorados mas sem cobrança externa. O Controller deve conhecer orçamentos e condições de parada da campanha. Nunca transforma uma permissão para testar localmente em autorização para consumir serviços pagos.

8. Coordenação, concorrência e intervenções do CLIENT

Num projeto com mais de um workstream, os owners devem distinguir quem decide, quem executa e que superfície de estado é canónica. Dois agentes podem alterar componentes independentes em paralelo; dois escritores sobre a mesma DB, schema ou contrato exigem coordenação explícita. O nome do projeto não é suficiente para

decidir se a concorrência é segura.

O I1 MASTER OWNER ajuda a resolver riscos de integração entre workstreams, duplicação de políticas, campanhas que não avançam e conflitos que o Planner local não consegue solucionar. Não deve transformar-se num segundo Planner que reabre todas as decisões, nem numa desculpa para que I3 devolva bugs de engenharia a um nível superior.

O CLIENT intervém nos pontos de decisão relevantes: confirmar uma regra, aprovar uma mudança de custo, avaliar a experiência ou aceitar o resultado. Pode ser um intermediário que cola uma mensagem completa entre ChatGPT e Codex se o ambiente não permite integração direta. O sistema continua responsável por preparar o contexto técnico e as verificações.

Uma comunicação operacional ao Controller deve ser completa e executável: objetivo, fontes atuais, limites, gates e critério de terminal. Não peça ao CLIENT que junte cinco patches nem que sincronize manuais de estado contraditórios. A organização existe precisamente para reduzir esse trabalho de coordenação humana.

9. Revisão independente e candidatos exatos

Antes de apresentar uma capacidade para aceitação, I3 integra um candidato definido e obtém os controles previstos. Um reviewer independente deve examinar o mesmo código, dados e artefactos relevantes. Se o candidato muda depois da revisão, é necessário renovar os controles afetados, sem fingir que a aprovação antiga se aplica ao novo conteúdo.

O protocolo de freeze e integridade de terminal organiza essas relações. Ele não garante por si só o comportamento de uma aplicação externa: quando a prova física importa, é preciso abrir ou executar realmente o produto e observar os passos exigidos.

Se a revisão encontra um defeito dentro do charter, o Planner devolve a I3 o conjunto de falsificadores. Se encontra uma nova decisão de produto, pede confirmação ao CLIENT e avalia o impacto estratégico. Um terminal técnico não pode ultrapassar a autorização humana e um screenshot agradável não substitui prova de dados.

O êxito depende de coerência entre afirmação e evidência. Um `PASS` deve identificar o gate, candidato e âmbito, não ser usado como certificado universal. A revisão humana também deve distinguir gosto subjetivo de uma regra objetiva já confirmada.

10. Uma rotina curta e reutilizável

No início de cada sessão, recupere objetivo, autoridade, fonte, currentness e bloqueios. Durante o trabalho, preserve checkpoints, execute alterações dentro do charter e verifique os efeitos materiais. Antes de apresentar um terminal, integre o candidato, execute gates aplicáveis, obtenha revisão e escreva um relatório conciso com evidência rastreável.

Se a sessão for interrompida, conserve a tentativa, journal e estado. O próximo agente tem de conseguir retomar sem reproduzir erros ou inventar progresso. Se houver contradição, HOLD sobre o efeito dependente e investigação segura. Se o bug estiver dentro do scope, I3 corrige. Se a premissa de produto mudou, o Planner reentra no limite certo.

No fim de uma semana, o CLIENT deveria ouvir quais capacidades são realmente

utilizáveis, que risco está aberto e qual decisão precisa tomar. Se a resposta consiste apenas em listas de commits e horas, a arquitetura ainda não transformou atividade em resultado.

A seguir: capítulo 04, como solicitar mudanças sem perder foco nem criar trabalho duplicado.

Fontes e âmbito

- Bootstrap e estado atual
- Reconstrução de currentness
- Integridade de freeze e revisão
- Ferramenta de delta
- Inventário e garbage collection
- Validação executável

Estes procedimentos descrevem responsabilidades e limites; não significam que qualquer host proprietário ou serviço de IA tenha sido integrado e testado pelo starter.

04 — Solicitar mudanças e novas capacidades

> Um pedido de alteração deve comunicar **o comportamento de produto pretendido**, o contexto e a forma de comprovar o resultado. O CLIENT não tem de desenhar a implementação. O Planner avalia decisões novas e impacto estratégico; I3 resolve defeitos e alterações técnicas dentro da campanha já autorizada.

Neste capítulo

1. Descrever o resultado que se quer obter
2. Distinguir novo requisito, bug e refactor
3. Um formato simples para pedir alterações
4. Prioridades, orçamento e critical path
5. Alterações que afetam dados e segurança
6. Feedback sobre UX e aceitação humana
7. Devolução de defeitos ao mesmo Controller
8. Como evitar campanhas e políticas duplicadas
9. Exemplos de pedidos bem delimitados
10. Confirmar que a alteração ficou atual

1. Descrever o resultado que se quer obter

Um utilizador não deveria precisar de escrever um mini-charter técnico para pedir algo útil. Pode dizer: «Quero encontrar as mensalidades vencidas por associado e ver a data do último pagamento». A frase descreve uma tarefa observável. O Planner aprofunda as regras que faltam, identifica dependências e decide onde essa capacidade se encaixa no produto.

Uma instrução como «melhora o backend» é muito ampla. Não explica qual problema existe nem que comportamento provar. «A pesquisa de associados demora demasiado quando temos cinco mil registos; quero obter resultados corretos sem esperar muito» é mais útil porque permite investigar desempenho real, ambiente e compromisso aceitável. Não é necessário impor previamente índices, caching ou framework.

As decisões funcionais pertencem ao CLIENT. Um agente pode propor opções, exemplos e trade-offs, mas não pode apresentar a sua preferência como requisito aprovado. Engenharia decide detalhes de implementação dentro dos limites confirmados. Essa separação permite que alguém sem experiência técnica participe plenamente na direção do produto.

O pedido deve indicar o que acontece hoje, o que deveria acontecer, quem é afetado e como saberíamos que foi corrigido. Se existir um vídeo ou captura demonstrando o problema, pode ser útil desde que não exponha dados confidenciais e esteja vinculado ao candidato real. Uma descrição clara vale mais do que uma grande quantidade de screenshots sem contexto.

2. Distinguir novo requisito, bug e refactor

Três situações podem parecer parecidas na conversa, mas têm owners e rotas diferentes. Um **novo requisito** altera o comportamento desejado. Um **bug** viola um comportamento já confirmado. Um **refactor** modifica estrutura técnica sem alterar a regra de produto prometida.

Suponha que o clube acordou que desativar um associado mantém o histórico de pagamentos. O sistema apaga esse histórico: é um bug dentro do charter. I3 corrige a persistência, testa e submete o novo candidato. Não precisa de uma campanha estratégica nova para «corrigir histórico».

Agora o CLIENT pede reativação de associados anteriormente desativados. Se a regra nunca foi definida, existe decisão nova: quem pode reativar, que identidade regressa e como ficam quotas em atraso. O Planner confirma os detalhes e avalia o impacto. Não é correto declarar que qualquer comportamento anterior era defeito sem um requisito que o estabeleça.

Um refactor, como dividir uma classe muito grande em componentes, pode estar ao alcance de I3 se serve o charter e preserva contratos. Se afetar interfaces públicas, migrações de dados ou compatibilidade de um downstream, a mudança pode tornar-se material. A classificação depende de efeitos e autoridade, não da etiqueta escolhida pelo programador.

Situação	Rota habitual
A funcionalidade já aprovada falha num caso.	I3 investiga e corrige na mesma campanha.
O CLIENT pede comportamento novo.	Planner confirma requisito e impacto antes de autorizar.
O código é reorganizado sem mudar a capacidade.	I3 executa com testes e revisão dentro do charter.
Um problema revela inconsistência entre produtos ou owners.	Coordenação estratégica apropriada, possivelmente I1.
Um serviço externo exige nova despesa ou permissão.	Aprovação reservada antes de realizar o efeito.

3. Um formato simples para pedir alterações

Não é preciso um formulário complicado. Para muitas mudanças basta comunicar cinco elementos: resultado observado hoje, resultado esperado, contexto, restrições e um critério concreto de aceitação. O Planner transforma essa informação numa decisão e num charter de capacidade quando necessário.

Por exemplo: «Na lista de associados, uma pessoa inativa aparece como ativa. Quero que a pesquisa de ativos a exclua, mas que os pagamentos anteriores continuem disponíveis a administradores. A correção está feita quando a pesquisa e o readback após reinício confirmarem essa regra no ambiente real». Se a regra já constava do charter, isso é uma descrição de bug diretamente executável.

Outro exemplo: «Na próxima versão quero poder exportar um relatório mensal em CSV para a tesouraria; deve incluir totais, não números de telefone». É uma capacidade nova. O Planner confirma período, formato, permissões, dados sensíveis e impacto na DB. Engenharia decide como gerar CSV, testes e estrutura de exportação, respeitando as restrições.

O CLIENT pode também fornecer referências visuais e preferências de experiência: «A tela de reservas deve mostrar vagas disponíveis antes de pedir os dados pessoais». A equipa testa a viabilidade e, quando a UX importa, apresenta opções para avaliação humana. Não se devem inventar gostos, aprovações ou objetivos comerciais com base em padrões genéricos.

4. Prioridades, orçamento e critical path

Mudar prioridade não é necessariamente mudar requisitos. Uma funcionalidade aprovada pode ser adiada por tempo ou capacidade, mantendo-se válida. Já alterar uma regra funcional exige atualizar a fonte canónica e avaliar efeitos nos módulos dependentes. Uma boa gestão distingue calendário, escopo e comportamento.

O Planner deve perceber o critical path real. Se inscrever membros depende do estado das quotas, não faz sentido declarar a campanha de inscrições independente de dados de pagamento ainda provisórios. Também não se deve impedir todo o trabalho caso existam componentes demonstravelmente independentes que possam avançar sem risco de integração.

Quando há prazo curto, a primeira entrega deve ser a menor capacidade que fornece valor comprovável. Um CRUD simples e durável pode ser melhor prova de conceito do que uma arquitetura de microserviços com os endpoints principais ainda falsos. Essa escolha não significa aceitar qualidade baixa, mas proteger o resultado observável.

O custo precisa de contexto: horas de desenvolvimento, infraestrutura, dados, revisão humana, operação e serviços externos. O número de agentes ou tokens não mede valor. Um mecanismo novo tem de desbloquear capacidade ou reduzir risco material; caso contrário aumenta complexidade e custo sem melhorar o produto.

Uma mudança que exige cobrança, publicação ou manipulação irreversível de dados não pode ser executada apenas porque o agente achou uma solução. O CLIENT ou responsável autorizado deve aprovar o efeito específico. O Planner documenta a decisão; I3 prepara preflight e executa apenas dentro dos limites concedidos.

5. Alterações que afetam dados e segurança

Algumas mudanças parecem pequenas na interface, mas têm consequências profundas. «Eliminar associados duplicados» pode apagar histórico legítimo. «Mostrar relatórios para todos» pode expor dados pessoais. «Atualizar a versão da DB» pode quebrar migrações ou backups. A forma correta de avaliar depende de dados, lineage, owners e recuperação.

O Planner reconstrói entradas e saídas, fontes de verdade, consumidores downstream, semântica de invalidation e gates. Engenharia identifica migrações, relações, locks, idempotência e rollback técnico. Se existir um snapshot SQLite/WAL, deve ser transacionalmente consistente; caso contrário a leitura de currentness pode estar errada.

Um pedido de remoção de dados precisa distinguir eliminação lógica, desativação, retenção obrigatória e apagamento físico. O CLIENT confirma a regra e permissões. O Controller não deve usar scripts destrutivos amplos sem resolver o alvo exato e sem proteger dados não abrangidos pela autorização.

O mesmo princípio aplica-se a artefactos. Um PDF de relatório ou um render de vídeo pode depender de fonte, configuração, licença e versão. Se um input relevante mudou,

a validade de artefactos derivados pode ter de ser revista. Não basta renomear um ficheiro como `final` para o tornar atual.

Segurança proporcional significa avaliar a exposição e o impacto reais. Um protótipo privado local não exige os mesmos mecanismos de uma plataforma pública com pagamentos, mas ambos precisam de respeitar segredos e não publicar dados confidenciais sem autorização.

6. Feedback sobre UX e aceitação humana

Uma pessoa pode achar uma interface difícil de usar apesar de todos os testes automatizados estarem verdes. A avaliação perceptual é um gate distinto. O feedback útil especifica a tarefa, o público e o problema observado: «Quando uma reserva falha por falta de vagas, o aviso não explica o que fazer» é mais concreto do que «a interface está feia».

Preferências visuais legítimas pertencem ao CLIENT ou a avaliadores humanos apropriados. Engenharia pode propor alternativas e explicar trade-offs, mas não deve assumir que a primeira composição aprovada num teste sintético é o design final. A demonstração deve utilizar o candidato exato, com dados fictícios e ambiente representativo.

Um comentário de UX pode ser um bug se contraria um comportamento já confirmado; pode ser uma mudança de produto se acrescenta uma regra nova; ou pode ser um ajuste em envelope de implementação. Classificar corretamente evita que cada pedido de alinhamento visual gere um novo planejamento estratégico.

Ao recolher feedback, proteja privacidade. Capturas com nomes, emails, tokens e informação de clientes não devem ser colocadas em PRs ou issues públicos. Um exemplo sintético pode reproduzir o problema sem divulgar dados reais. A revisão humana deve dizer o que viu, não atribuir aprovações a quem não participou.

7. Devolução de defeitos ao mesmo Controller

Quando o Planner rejeita um terminal por problemas dentro do charter, devolve a I3 um conjunto **consolidado de falsificadores**. Cada item explica esperado, observado, candidato, evidência e gate. O Controller pesquisa a causa, corrige, testa e integra um novo candidato com revisão independente adequada.

Não se cria nova campanha, novo charter nem PASS extra porque uma fixture falhou. Esse tipo de churn faz o Planner trabalhar como um depurador de primeira linha e obriga o CLIENT a acompanhar engenharia que já devia ter ownership operacional. O charter permanece vigente até existir aceitação estratégica real ou invalidation material.

Se o mesmo subsistema falhar repetidamente, I3 deve interromper a sequência de microcorreções e realizar hardening abrangente. Isso pode implicar estudar as transações, procurar falhas semelhantes, reforçar testes de regressão e usar revisão independente. O objetivo é eliminar a classe de defeitos, não colecionar terminais parcialmente verdes.

Uma mudança de comportamento pedida pelo CLIENT não deve ser escondida como bug fix. Do mesmo modo, um erro ordinário não deve ser inflacionado para invalidation estratégico. A diferença protege a velocidade e a autoridade de quem decide o produto.

8. Como evitar campanhas e políticas duplicadas

O sistema funciona melhor quando cada preocupação tem um owner canónico e uma superfície atual. Se duas campanhas editam o mesmo recurso sem coordenação, podem gerar referências incompatíveis. Se dois documentos dizem qual é o estado `CURRENT`, uma sessão nova pode escolher o errado.

Antes de criar um ficheiro ou política nova, procure o contrato existente. Uma atualização pequena ao lugar correto é preferível a outra cópia de verdade. As decisões de produto, estado mutável, autoridade de campanha e transporte/runtime devem permanecer conceitualmente separados, embora estejam ligados por referências verificáveis.

Reaproveitar artefactos também exige lineage e identidade. Se um relatório aprovado é reutilizado numa nova feature, registe a relação e invalide quando inputs relevantes mudarem. A contagem de documentos e commits não é prova de que essa rede de referências esteja coerente.

Um MASTER OWNER deve detetar churn sistémico, duplicação de políticas e falhas de integração entre workstreams. A solução apropriada é corrigir o owner e o processo que causam a repetição, não adicionar mais um nível de aprovação a cada bug.

9. Exemplos de pedidos bem delimitados

Caso A — Filtro de mensalidades. «Preciso identificar associados que não pagaram o mês atual. O cálculo deve considerar pagamentos parciais e permitir verificar o histórico». O Planner pergunta pela definição de período e pagamento parcial, confirma a regra e delimita consultas, permissões e testes. I3 implementa com fontes de dados atuais.

Caso B — Corrigir perda de informação. «Quando desativo um associado, os pagamentos desaparecem após reiniciar. O requisito era preservá-los». Trata-se de bug in-charter. I3 investiga persistência e restaura o comportamento, mantendo evidência do candidato anterior e do novo.

Caso C — Novo relatório CSV. «A tesouraria necessita exportar totais por mês sem dados pessoais». É nova capacidade. O Planner define formato, público, autorização e limites. Engenharia escolhe geração, casos negativos e cuidado com formula injection quando pertinente.

Caso D — Melhorar uma tela. «O botão de cancelar reserva fica oculto no telemóvel e o cliente não consegue concluir a tarefa». É necessário verificar o layout no dispositivo relevante e avaliar se a alteração se encontra no envelope já autorizado. Uma captura isolada em desktop não prova a correção.

Caso E — Migrar base de dados. «Queremos passar de armazenamento local para um servidor partilhado». A mudança tem implicações de rede, backup, custos, concorrência e segurança; requer avaliação estratégica, plano de migração, critérios de rollback e provas de conservação de dados.

O que estes exemplos partilham não é um template rígido de prompts. É a separação entre a intenção humana, as decisões ainda abertas, o trabalho técnico e a evidência exigida para afirmar que o resultado foi alcançado.

10. Confirmar que a alteração ficou atual

Depois de integrar um candidato, não basta mostrar um diff ou repetir «feito». É necessário verificar que o comportamento exigido funciona no ambiente relevante e que Product State, fonte, DB e artefactos afetados correspondem ao candidato submetido. A revisão deve ser realizada sobre essa identidade, não sobre outra versão com nome parecido.

O Planner audita o terminal e regista a decisão apropriada. O CLIENT faz a aceitação que lhe compete, por exemplo da experiência visual. Se um downstream depende da nova regra, é preciso verificar que a referência para a capacidade aceite está atual antes de iniciar trabalho dependente.

Uma sessão futura deve recuperar essa decisão sem consultar o chat original. Se encontrar dois requisitos contraditórios, um link quebrado ou evidência de um candidato antigo, não inventa continuidade: reconcilia ou declara HOLD concreto. A qualidade da mudança inclui a capacidade de ser compreendida e mantida posteriormente.

Um pedido de alteração bem realizado termina com menos incerteza sobre o produto, não apenas mais código. O sistema deve deixar claro o que mudou, porquê, quem aprovou a regra, o que foi testado e que próximo passo está autorizado.

A seguir: capítulo 05, erros, incidentes e recuperação sem perder a verdade do projeto.

Fontes e âmbito

- Limite das decisões do CLIENT
- Padrão de charter de engenharia
- Ciclo de capacidades e evidência
- Requisitos, stack e plano de entrega
- Integridade de revisão e terminal
- Primeiro uso v1.4

Os pedidos fictícios não criam requisitos nem campanhas reais. O produto e as fontes atuais determinam a autoridade e a evidência necessária.

05 — Erros, incidentes e recuperação

ArquitecturaRepo · Manual completo de utilização · Edição v1.4 · Português

> Recuperar um projeto significa voltar a um estado **verdadeiro e verificável**, não apenas eliminar um erro do terminal. Preserve fonte, ownership, journal de transações, DB e evidência atual. Um bug comum dentro de um charter pertence a I3; uma contradição de autoridade ou currentness exige reconciliação apropriada.

Neste capítulo

1. Preservar a evidência antes de reparar
2. Falhas do ambiente, contas e permissões
3. Quando a camada de controlo não valida
4. Git não é uma forma de apagar problemas
5. Repetição de defeitos e causas sistémicas
6. Segurança, privacidade e direitos externos
7. Provar a saída de um incidente
8. Três cenários de recuperação
9. Operações sensíveis e limites de autoridade
10. O que uma validação verde não demonstra
11. Feedback e dados locais do starter

1. Preservar a evidência antes de reparar

Quando ocorre um problema, guarde a mensagem exata, o comando ou ação que o provocou, data, diretório, fonte/branch, candidato e eventuais operações pendentes. Não apague por reflexo a pasta `.agentic-sdlc/`, a base de dados ou os logs. Uma reinstalação pode destruir precisamente a informação necessária para perceber o que aconteceu.

Primeiro classifique a falha. Pode ser uma ferramenta ausente, um problema de permissões, um bug da aplicação, um contrato de governação inválido, fonte antiga, uma transação incompleta, um fornecedor externo indisponível ou uma autorização do CLIENT ainda pendente. Essas causas têm owners e soluções diferentes.

Sinal	Resposta inicial segura
Uma prova de integração de uma feature falha.	I3 conserva a reprodução e corrige na campanha vigente.
O validador rejeita uma referência canónica.	HOLD dependente, identificar relação, fonte e owner causais.
Git mostra mudanças locais e remotas incompatíveis.	Preservar ambas, estudar o diff e reconciliar.
Dados reais podem estar danificados.	Suspender escritas sensíveis, proteger backups, WAL e journals.
Falta permissão para publicar ou gastar.	Não executar o efeito até obter autorização específica.

Sinal	Resposta inicial segura
Um host caiu durante uma operação.	Efeito desconhecido; fazer readback antes de reexecutar.

Uma correção tem de resolver a causa e demonstrar o gate afetado. Uma melhoria cosmética que muda o estado de vermelho para verde pode deixar o produto defeituoso. Enquanto escritas materiais permanecem suspensas, muitas vezes é possível continuar a investigação read-only e preparar testes em ambiente isolado.

O CLIENT precisa de compreender a consequência do incidente e as decisões que lhe cabem, não a implementação de cada script de recuperação. O Controller é responsável por reproduzir e corrigir falhas dentro da sua autoridade. Uma escalada estratégica é necessária quando os limites do charter foram realmente invalidados.

2. Falhas do ambiente, contas e permissões

Python ou comando não encontrado. Confirme a versão, localização do executável, PATH e diretório de trabalho. Instale dependências declaradas, como `jsonschema`, usando fontes confiáveis. Não modifique um schema canónico para contornar uma dependência local ausente.

Não consegue ler GitHub. Diferencie autenticação, âmbito do token, visibilidade do repositório, instalação do conector e acesso à branch. Uma falha de leitura não autoriza a entregar credenciais em texto simples nem a ampliar permissões para toda a conta. Teste somente o acesso necessário e preserve a mensagem de erro.

O agente abriu o projeto errado. Compare caminho, branch, origem e ficheiros antes de escrever. Duas pastas com nomes semelhantes podem ter histórias diferentes. Se o runtime apontar para outro worktree, registe o desvio e volte a reconstruir a fonte atual de forma segura.

Ocorreu timeout ou limite de utilização. A sessão de um serviço não define a identidade da campanha. Goal, charter, candidatos, owner, checkpoints e ações pendentes pertencem ao estado durable. Um novo agente deve reconstruí-los sem inventar autorização. A falha do host não transforma automaticamente uma operação em «não executada».

Permissão negada no sistema de ficheiros. Descubra o alvo exato e porquê a operação requer escrita. Não tente contornar proteções para remover um diretório amplo. Se o contrato de segurança exige aprovação, pare a operação e obtenha-a, em vez de procurar outro meio de fazer o mesmo efeito sem permissão.

O tratamento de ambiente deve ser proporcional. Muitos problemas resolvem-se com um diagnóstico simples; outros revelam estado parcial que precisa de recovery. O que importa é não confundir a primeira mensagem de erro com a causa final.

3. Quando a camada de controlo não valida

O `tools/validate_project.py` instalado verifica contratos e relações de `.agentic-sdlc/`. Pode detetar ownership duplicado, referências CURRENT inconsistentes, candidato e revisão incompatíveis ou mudanças de schema incompletas. Não prova por si só que a aplicação abra, que um login funcione ou que a base de produção contenha os dados esperados.

Antes de alterar estado, guarde o erro, root de controlo, ref Git, alterações locais e testigos de DB. Investigue se falta uma dependência, se um owner foi supersedido, se há journal pendente ou se um terceiro modificou um destino. Não coloque `READY` manualmente num JSON nem apague o journal para obter um relatório verde.

As transições podem afetar vários ficheiros canónicos. O protocolo de recuperação transacional trabalha com hashes anteriores e posteriores e rejeita conflitos. Se a recuperação descobre que um ficheiro foi alterado por outro processo, deve preservar `CONFLICT`, não sobrescrever por comodidade. Depois é preciso validar os efeitos físicos e atualizar referências coerentes.

Em SQLite com Write-Ahead Logging, os bytes do ficheiro principal `.db` podem não conter todos os writes confirmados. É necessário um snapshot transacionalmente consistente quando o estado atual depende dele. A rota de attachment de governação existente aborda esse testemunho; não é uma ferramenta automática de reparação da base de produção.

Se o teste de estrutura passa mas o Product State aponta para um candidate antigo, ainda existe problema de currentness. Se o estado é coerente mas a aplicação não arranca, existe problema físico. Os gates são distintos e não se substituem mutuamente. Um bom diagnóstico explica qual passou e qual continua pendente.

4. Git não é uma forma de apagar problemas

Quando Git mostra um conflito, não escolha cegamente «minha versão» ou «versão remota». Ambas podem conter trabalho útil. Investigue a base comum, diff local, alterações remotas e ownership do recurso. Uma estratégia de merge tem de preservar significado, não apenas produzir um repositório com status limpo.

Não use resets destrutivos ou elimine um checkout para escapar ao trabalho de conciliação. Se existirem ficheiros do utilizador, estado de uma campanha ou dados derivados que continuam a ter valor, devem ser preservados até ser possível provar que a limpeza não afeta autoridade nem evidência.

O starter fornece inspeção incremental de delta e inventário/limpeza de artefactos. Essas ferramentas não autorizam apagar código, dados humanos desconhecidos ou artefactos aceites. Uma operação de garbage collection começa por listar alvos e validar ownership; apenas recursos descartáveis e explicitamente identificados podem ser removidos.

Um nome de branch não prova quem produziu um commit, nem qual campanha o autorizou. A identidade do ator e a autoridade de execução devem vir de registos próprios. Se dois agentes afirmarem ter escrito a mesma mudança, o histórico Git pode ajudar a investigar, mas não substitui a cadeia de autorização.

Quando um ficheiro de configuração mudou depois da revisão, o reviewer deve avaliar o novo candidato na parte afetada. Um checkout limpo com código novo não transforma uma revisão anterior em evidência atual. A disciplina de fonte exata é tão importante na recuperação como na aceitação.

5. Repetição de defeitos e causas sistémicas

Um bug previsto pelo charter pertence a I3. Isso inclui testes falhados, correções de integração, pequenos refactors, recuperação de crashes e auditorias técnicas dentro do scope. Não há motivo para criar uma campanha nova ou pedir ao Planner um PASS estratégico adicional depois de cada defeito.

Se o mesmo tipo de problema regressa várias vezes, a resposta muda. Em vez de corrigir cada sintoma isoladamente e declarar novo terminal prematuro, o Controller investiga a classe inteira: contratos, dados, concorrência, testes, boundaries e revisões anteriores. Realiza hardening abrangente, novas provas relevantes e revisão independente.

Considere um módulo que perde pagamentos ao desativar associados. Se um patch resolve um caso mas continua a falhar com pagamentos parciais, não é suficiente trocar mais uma condição. É necessário compreender o modelo de persistência, relações, transações e regras de retenção, com casos negativos representativos e readback real.

O Planner volta quando existe terminal estratégico adequado ou evidência nova que invalida o charter. Aumentar a dificuldade de implementação não significa por si só que o produto tenha mudado. A cadeia de ownership protege a execução de ficar presa num ciclo de replaneamento.

O MASTER OWNER também deve reconhecer problemas sistémicos de governação: policies duplicadas, Controllers que devolvem bugs repetidamente, integrações entre workstreams com owner indefinido ou deadlines irrealistas. Nesse caso a correção recai sobre a causa do sistema, não sobre criar novas microcampanhas.

6. Segurança, privacidade e direitos externos

Os logs e exemplos públicos devem ser sanitizados. Não inclua tokens, senhas, nomes completos de clientes, caminhos pessoais, bases privadas, documentos de terceiros nem transcrições de conversas em issues ou relatórios que saem do projeto. Um defeito costuma poder ser reproduzido com dados fictícios e uma descrição técnica precisa.

A instalação do framework não autoriza o envio de código a fornecedores externos. Modelos e serviços são opcionais, e credenciais dependem do ambiente e de decisões específicas. Se uma solução exige um serviço pago, uma ligação de rede nova ou publicar algo, obtenha aprovação apropriada antes de executar o efeito.

As licenças devem ser proporcionais ao uso. O facto de o starter ser MIT não autoriza qualquer imagem, fonte, música ou dataset utilizado pela aplicação. Da mesma forma, a ausência de uma ficha documental perfeita não significa automaticamente que um recurso seja proibido: avalie direitos concretos e evidência mínima razoável sem burocracia artificial.

Para recuperação de dados, distinga leitura, restauração, sobrescrita e apagamento. Um backup pode conter informação sensível; guardá-lo em local público não é uma forma aceitável de facilitar um teste. A engenharia tem de preservar confidencialidade e integridade durante toda a recuperação.

7. Provar a saída de um incidente

Uma recuperação só fica concluída quando os efeitos são verificados contra a fonte atual. O relatório deve indicar incidente, candidato, ambiente, causa demonstrada, alterações realizadas, testes repetidos, estado da DB e próximos riscos. O desaparecimento de um stack trace não basta se o utilizador ainda não consegue realizar a tarefa.

Quando se corrige uma migração, faça leitura posterior e, se necessário, reinício.

Quando se corrige um problema de UI, abra a aplicação no ambiente relevante. Quando um host caiu a meio de uma chamada externa, consulte o efeito remoto antes de tentar novamente. Uma segunda execução cega pode duplicar pagamentos, publicações ou alterações de estado.

Se a recuperação exigiu modificar o candidato depois de uma revisão, renove a parte afetada da revisão. A decisão final pertence ao owner competente. I3 pode declarar um terminal técnico com evidência, mas não pode aprovar sozinho a experiência reservada ao CLIENT nem conceder autorizações externas.

Um resumo de incidente deve ser suficientemente curto para orientar a pessoa que retoma o trabalho e suficientemente preciso para localizar a evidência. A continuidade não necessita de transcrições enormes, mas precisa de referências corretas e owners claros.

8. Três cenários de recuperação

Cenário A — Instalação interrompida. O instalador escreve parte de `.agentic-sdlc/` e o terminal fecha. Antes de repetir, inspecione o destino e descubra se há estado parcial. Preserve ficheiros existentes, compare com o manifesto e utilize o caminho de reparação ou reinstalação segura autorizado. Não apague toda a pasta do projeto.

Cenário B — Operação de DB incerta. Um pedido de pagamento recebe timeout. O cliente não sabe se o valor foi registado. O Controller procura um identificador de operação e faz readback da fonte transacional antes de reexecutar. Se não conseguir determinar o efeito com segurança, suspende reenvios que possam duplicá-lo e estabelece HOLD específico enquanto investiga.

Cenário C — Revisão desatualizada. O reviewer aprovou versão A, mas um refactor produziu B. O Planner não aceita B com base no PASS de A. I3 identifica o delta, executa os gates afetados e apresenta uma revisão aplicável a B. A evidência não afetada pode continuar válida se a independência estiver justificada.

Cada caso ilustra um tipo diferente de risco: efeitos parciais, idempotência e identidade do candidato. A resposta correta não consiste em reiniciar o processo estratégico; consiste em preservar a verdade e reparar no owner adequado.

9. Operações sensíveis e limites de autoridade

Antes de realizar uma ação de risco, pergunte: qual é o alvo exato, de quem é o recurso, quais os efeitos reversíveis, que dados podem ser expostos, qual o rollback e que autorização existe? A resposta deve ser baseada no sistema atual, não em uma suposição de que «isto é só um teste».

Operação	Limite relevante
Ler fontes e executar testes locais.	Escopo e permissões técnicas habituais.
Modificar código dentro de um charter.	Ownership, gates e revisão de I3.
Alterar schema e dados reais.	Preflight, migração e recuperação verificáveis.
Apagar diretórios ou bases.	Alvo específico e autorização expressa quando material.

Operação	Limite relevante
Enviar ficheiros a terceiros.	Privacidade, licença, custo e permissão do CLIENT.
Publicar release ou comunicar externamente.	Gate de release e autoridade externa aplicável.

Quando falta a autorização, pare o efeito e peça-a. Não procure uma rota alternativa para contornar um bloqueio. Quando o problema é apenas uma falha técnica dentro do contrato, I3 deve resolvê-lo sem transferir decisões internas ao CLIENT.

Os limites também se aplicam à recuperação automática. Um script que limpa caches pode ser razoável, mas não deve remover ficheiros desconhecidos numa pasta ampla. Um mecanismo que repete transações precisa de idempotência comprovada. A capacidade de automatizar uma operação não justifica executá-la sem conhecer o seu impacto.

10. O que uma validação verde não demonstra

O `validate_structure.py` do repositório verifica relações documentais e contratos estruturais. Um teste de scaffold confirma que a camada pode ser instalada em cenários controlados. Uma prova de recuperação sintética verifica o comportamento de um mecanismo sob determinadas condições. Nenhuma dessas provas certifica uma aplicação de terceiros em produção.

Um teste unitário pode passar enquanto a API está mal ligada; uma integração pode passar enquanto o estado de DB aponta para uma versão errada; uma aplicação pode funcionar fisicamente mas ter experiência confusa. O gate semântico e o gate humano têm responsabilidades próprias. O relatório deve distinguir o que foi observado do que ainda depende de prova.

As releases versionadas têm outra fronteira: para dizer que um ZIP foi publicado, é necessário verificar o artefacto exato no GitHub Release e os checksums correspondentes. Um build local e uma tag não provam a disponibilidade remota de todos os ficheiros. E uma release do framework não é uma release de cada produto que o utiliza.

Se um gate não é aplicável, explique porquê de forma proporcional. Se é aplicável e falta prova, não o substitua por outro que já passou. O objetivo é evitar tanto falsos positivos como um processo infinito de testes irrelevantes.

11. Feedback e dados locais do starter

Na v1.4, a geração local de relatórios mínimos de metadados pode estar ativa por padrão. O workflow periódico, quando instalado, guarda a saída nos artefactos GitHub Actions do próprio projeto. A distribuição pública não inclui um endpoint de coleta externo. A opção `--no-feedback-workflow` desativa o agendamento, não a geração local.

Para desligar inteiramente a geração, utilize `enabled=false` em `.agentic-sdlc/feedback/ARCHITE`
Antes de ativar uma integração outbound, verifique a política de feedback e privacidade, o código do workflow e as permissões reais. Não presuma que uma opção de privacidade na UI altera automaticamente o comportamento de todos os scripts.

Se precisar reportar um incidente do próprio framework, inclua versão, plataforma, passos reproduzíveis, logs sanitizados e resultado observado. Não anexe os dados privados da aplicação. Uma descrição baseada em um projeto sintético pode permitir a correção sem expor informação sensível.

O objetivo final da recuperação é que qualquer trabalhador autorizado possa continuar de um estado coerente, sabendo o que passou, o que falhou e o que permanece por decidir. Um sistema que só parece recuperado enquanto a conversa original continua aberta precisa de mais evidência durable.

A seguir: capítulo 06, fechamento de capacidades e aceitação com candidatos exatos.

Fontes e âmbito

- Recuperação de transações
- Attachment de governação existente
- Ferramenta de delta de fonte
- Inventário e GC de ficheiros
- Privacidade e feedback
- Verificação pública v1.4

As soluções exatas dependem do ambiente e do produto. Nunca trate um exemplo de recuperação como autorização para substituir ou apagar estado canónico real.

06 — Encerramento de capacidades e aceitação

ArquitecturaRepo · Manual completo de utilização · Edição v1.4 · Português

> Uma capacidade fecha contra **um candidato exato e a evidência aplicável**. O terminal de I3 convida à auditoria estratégica; não aprova automaticamente o produto, não substitui o gate humano e não autoriza publicação ou deploy. Este capítulo distingue as decisões e explica como devolver erros à mesma campanha sem recomeçar a arquitetura.

Neste capítulo

1. A capacidade é um resultado de produto
2. Cinco componentes do encerramento
3. Candidato exato como unidade de revisão
4. Evidência insuficiente sem campanhas artificiais
5. Aceitar, retomar ou mudar o objetivo
6. Revisões periódicas e regra de paragem
7. Registo de encerramento auditável
8. Exemplo de rejeição e correção
9. Produto utilizável e framework verde
10. O que fazer depois do encerramento

1. A capacidade é um resultado de produto

«Gestão de associados» não está concluída quando alguém desenha um formulário nem quando um agente cria commits. Está concluída quando a pessoa prevista consegue realizar as ações acordadas no ambiente relevante, os dados se mantêm como devem, os testes aplicáveis passam, o estado atual é coerente e as autoridades competentes aceitam o resultado.

O Planner define fronteiras e charter; I3 implementa, integra um candidato, produz evidência e obtém revisão independente quando necessária. O Planner audita o terminal segundo os requisitos atuais. O CLIENT aceita preferências, comportamento do produto e qualidade perceptual dentro da sua autoridade. Estes papéis não se substituem só porque um deles declara `PASS`.

Uma campanha não precisa durar um número fixo de prompts. Pode sobreviver a limites de um serviço, falhas do ambiente, reintentos e checkpoints. Isso não gera nova autorização nem apaga a evidência anterior. Por outro lado, muitas horas de trabalho e um relatório confiante não demonstram que alguém consegue utilizar a funcionalidade.

Um protótipo com screenshots pode mostrar boa direção visual; ainda não prova persistência, permissões ou recuperação após reinício. Uma API que responde a um endpoint pode precisar de testes de integração e dados. A aceitação deve corresponder ao resultado de produto que o CLIENT realmente solicitou.

2. Cinco componentes do encerramento

Componente	Pergunta essencial
Resultado visível	A pessoa prevista consegue concluir a tarefa no ambiente relevante?
Verificações aplicáveis	Foram realizados os controles de lógica, integração, dados, segurança e experiência pertinentes?
Estado atual coerente	Código, configuração, requisitos e dados/artefactos apontam para o mesmo candidato?
Sem bloqueio material oculto	Algum defeito aberto invalida o resultado anunciado?
Aceitação competente	Planner e CLIENT examinaram a evidência que lhes compete?

Nem toda capacidade exige todos os testes imagináveis. Um script interno simples pode dispensar avaliação estética. Uma operação com mensalidades exige persistência e um percurso de utilização confiável. A regra é que um gate não substitua silenciosamente outro relevante.

O vocabulário de gates ajuda a evitar confusão:

- **Unit:** funções ou regras isoladas comportam-se corretamente em entradas controladas.
- **Integration:** componentes e interfaces funcionam juntos, incluindo casos negativos.
- **DB/currentness:** dados, migrações, snapshots e referências atuais são coerentes e duráveis no âmbito exigido.
- **Semantic/generalization:** o comportamento corresponde à regra de negócio, não apenas a uma fixture conveniente.
- **Physical:** o programa, artefacto ou percurso executa realmente no ambiente adequado.
- **Perceptual/human:** uma pessoa competente avalia qualidade subjetiva e experiência quando importa.
- **Release:** a distribuição exata, direitos, permissões, hashes e condições de publicação estão satisfeitos.

Um teste unitário verde não demonstra que a UI abre; um JSON válido não prova CRUD; um screenshot agradável não prova que o banco persiste. O gate precisa de candidato, ambiente, resultado e owner claros. Acrescentar controles que não reduzem risco é desperdício; omitir os que importam produz aceitação prematura.

3. Candidato exato como unidade de revisão

Um reviewer independente tem de examinar a mesma fonte e estado que serão submetidos à aceitação. Isso pode exigir commit, configuração, schema da DB, testigos de dados, hashes dos assets e ambiente. Uma revisão do candidato A não se transfere automaticamente ao candidato B só porque ambos usam a branch `main` ou um ficheiro chamado `final`.

A sequência normal é: integrar candidato, congelar identidade relevante, realizar testes físicos e revisão independente, apresentar terminal estratégico. Se alguém modifica materialmente a implementação após a revisão, renova os controles afetados. Evidência verdadeiramente independente da alteração pode ser reaproveitada quando a relação com as fontes é demonstrável.

O protocolo de review freeze e integridade de terminal especifica esta relação. O controlador portátil pode modelar estados e contratos, mas não pode fingir ter observado uma aplicação externa ou recebido autorização humana real.

Mesmo depois de um PASS técnico, podem continuar pendentes revisão de privacidade, disponibilidade em produção, aceitação perceptual, licença de terceiros ou autorização de release. Registe estes gates separadamente. Um terminal técnico significa «há evidência para o próximo julgamento», não «todos os efeitos posteriores estão autorizados».

Essa disciplina protege a própria equipa. Um revisor que encontra uma falha num candidato desatualizado não deve bloquear indevidamente uma versão diferente sem verificar o delta. E uma aprovação antiga não deve ser utilizada para encobrir uma mudança de comportamento posterior. A identidade exata permite determinar o que precisa realmente ser revisto.

4. Evidência insuficiente sem campanhas artificiais

Se um terminal falha por razões que já pertencem ao charter, o Planner devolve a I3 um conjunto **consolidado de falsificadores**: requisito esperado, comportamento observado, candidato, gate e evidência. I3 corrige, testa, integra e obtém revisão na mesma campanha. Não pede um charter novo porque uma transação falhou.

Uma falha de teste, defeito identificado por reviewer, migração em envelope, refactor interno ou crash recuperável são problemas de execução. Não se criam PASS4 ou PASS5 para transformar o Planner num depurador. O Controller tem autoridade e responsabilidade por essas reparações até existir um terminal estratégico.

Se o mesmo tipo de defeito regressa repetidamente, I3 investiga o subsistema completo. Pode ser necessário rever contratos, persistência, concorrência, testes e a integração. Uma correção que esconde apenas um sintoma não satisfaz a capacidade; o objetivo é reduzir a classe de risco de forma demonstrável.

Uma decisão nova de negócio, alteração de permissões ou incompatibilidade estratégica pode invalidar o charter. Nesse caso o Planner volta à estratégia. A distinção não depende de quão complicado ficou o código, mas de saber se o resultado continua dentro da autorização vigente.

Um terminal bem escrito contém candidato e ambiente, percurso observado, gates aprovados, gates abertos com impacto, riscos e próxima ação com owner. É um índice para fontes canónicas, não uma nova cópia do Product State. Se uma referência aponta para um log antigo, não serve para aceitar o estado presente.

5. Aceitar, retomar ou mudar o objetivo

Aceitar. O Planner audita a evidência pertinente e regista a capacidade aceite nas superfícies canónicas. O CLIENT realiza a sua aceitação perceptual ou de produto quando aplicável. Downstreams só devem depender de uma capacidade se os requisitos necessários foram efetivamente demonstrados.

Retomar após interrupção. Uma sessão nova reconstrói charter, owner, checkpoint, candidato e efeitos pendentes. Não declara a campanha terminada porque foi atingido um limite de tokens ou porque o host já não consegue ler uma conversa antiga. A fonte física continua mais forte que a memória do agente.

Mudar prioridades. Um atraso de cronograma não muda necessariamente a regra de negócio. Alterar a política de pagamento ou elegibilidade muda. Registe supersessões e dependências, preservando trabalho que continua válido. Não mantenha dois charters contraditórios como atuais para o mesmo concern.

Manter HOLD. Se não existe forma de relacionar um candidato com DB ou fonte atual, bloqueie apenas as ações dependentes da incerteza. Provas de leitura e investigação segura podem continuar. Um HOLD específico é melhor que inventar currentness ou paralisar indiscriminadamente o produto inteiro.

6. Revisões periódicas e regra de paragem

Uma reunião semanal deveria responder às seguintes perguntas: o que está aceite, o que pode ser demonstrado hoje, qual a prioridade seguinte, que prazo ainda é plausível, que risco importante existe e qual decisão é realmente do CLIENT. As respostas devem apontar para resultados físicos, não apenas para o volume de atividade.

Commits, documentação, número de agentes, tokens e horas de runtime são meios e custos. Podem ser necessários para entregar uma capacidade, mas não são a capacidade em si. Um projeto que passa semanas a produzir relatórios sem completar um percurso de utilizador precisa de rever o método, não celebrar a produção de documentação.

O framework tem uma regra de paragem: depois de satisfazer os gates de release, não deve continuar a ganhar políticas genéricas sem uma falha identificada ou benefício real. Cada novo mecanismo aumenta manutenção e possíveis contradições. A mesma disciplina vale para os produtos feitos com ele.

O equilíbrio deve ser concreto. Aceitar uma feature quebrada porque uma fixture passou é perigoso; bloquear indefinidamente uma feature válida por um microdefeito sem impacto é improdutivo. A revisão estratégica distingue riscos materiais de detalhes que podem ser melhorados depois, com as autoridades apropriadas.

7. Registo de encerramento auditável

Um trabalhador que entra amanhã deve conseguir localizar a evidência sem ler todo o chat. Um registo curto pode apontar para:

Campo	O que deve ser rastreável
Candidato	Fonte, configuração, estado e artefactos exatos revistos.
Resultado	Entrada, ação e saída observada no percurso end-to-end.
Testes	Comandos relevantes, ambiente e outputs.
Persistência	Readback, reinício e recuperação quando necessários.

Campo	O que deve ser rastreável
Segurança e direitos	Verificações proporcionais à privacidade e ao material externo.
Revisão	Reviewer, âmbito e identidade do candidato.
Mudanças posteriores	Delta após freeze e gates renovados.
Gate humano	Aprovação ou rejeição autêntica, nunca inferida.
Próxima rota	Aceitar, devolver a I3, perguntar ao CLIENT ou HOLD específico.

Não crie outro ficheiro `CURRENT` só para organizar essa tabela. Use links para as superfícies com owner canónico. Se um link falta, se a DB não corresponde à fonte ou se a prova é apenas uma fixture não relacionada com o produto, o registo ainda não sustenta a aceitação.

Também é necessário diferenciar provas do framework e provas da aplicação. A instalação e recuperação portátil do starter podem ser verificadas na CI, mas a experiência de um utilizador num projeto concreto necessita dos seus próprios testes. Um gate de release do framework não confere garantia universal a todas as aplicações que o adotarem.

8. Exemplo de rejeição e correção

Suponha que uma aplicação do clube permite criar e desativar associados. A revisão independente descobre que uma desativação elimina o histórico de pagamentos, embora o charter determinasse retenção. O Planner devolve a I3 um falsificador claro: «Desativar um associado com pagamentos anteriores deve preservá-los após reiniciar; o candidato X elimina-os».

I3 investiga relações e persistência, corrige a causa, acrescenta regressão significativa, testa o readback depois do reinício, integra novo candidato e pede revisão independente. Não abre uma campanha chamada «corrigir desativação» e não exige que o CLIENT escolha a foreign key apropriada.

Se a alteração produz B e o reviewer avaliou A, o PASS de A não aceita B. Pode reutilizar-se evidência de componentes não afetados, mas os testes de persistência e a revisão sobre B precisam ser renovados. Isso evita tanto repetição excessiva como aprovação desatualizada.

Agora imagine que o CLIENT pede uma funcionalidade de reativação com permissões especiais. É comportamento novo e precisa de uma decisão sobre quem pode reativar e como fica o histórico. O Planner avalia o impacto e a autorização. A nova regra não é a mesma coisa que corrigir um defeito de regra antiga.

9. Produto utilizável e framework verde

Uma campanha pode passar na validação de contratos do controller sem produzir uma aplicação utilizável. Para aceitar um CRUD de associados, abra a aplicação real, crie um registo fictício, pesquise, edite, desative e confirme a retenção de histórico. Se a persistência importa, reinicie e faça uma nova leitura. Se existem permissões, teste também o acesso que deve ser recusado.

O `tools/verify_v1_release.py` do ArquitecturaRepo verifica um snapshot concreto do framework e cenários sintéticos. Verificação de fonte, instalação portátil Linux/Windows, leitura de artefactos de release e aceitação de um produto de terceiros são gates diferentes. A documentação de suporte público v1.4 especifica o que foi exercitado.

É incorreto anunciar «todos os testes passaram» sem indicar de que testes se trata. Pode ser verdade que a camada de controlo valide e que a aplicação ainda esteja em `NOT RUN` no gate físico. Uma declaração honesta permite continuar o trabalho certo; uma frase global pode induzir o CLIENT a aceitar algo inexistente.

Uma integração nativa com outro host também exige prova própria. O funcionamento de scripts de referência não é prova de que Codex Desktop, um fornecedor pago ou um serviço externo tenham sido autenticados e executados com sucesso no ambiente adotado.

10. O que fazer depois do encerramento

Depois da aceitação, atualize Product State, lineage e referências atuais para o candidato correto. Desbloqueie apenas os dependentes justificados por essa aceitação. Se houver rejeição por bug dentro do charter, retorne a I3. Se uma mudança de produto invalidou a estratégia, volte ao Planner. Se falta prova de currentness, mantenha HOLD sobre o trabalho dependente.

O CLIENT deve conseguir terminar a sessão sabendo o que está realmente utilizável, o que continua aberto, quem resolve a questão e quando haverá nova demonstração. Se só o agente da conversa anterior consegue responder, a continuidade ainda é insuficiente.

Um bom encerramento dispensa um relatório gigantesco, mas exige identidade, evidência e decisão. A pergunta final é: alguém que não participou no desenvolvimento consegue confirmar a afirmação a partir dos artefactos e fontes apresentados? Se a resposta for não, ainda falta trabalho de engenharia ou de documentação da prova.

A seguir: capítulo 07, um exemplo completo de aplicação de gestão de um clube.

Fontes e âmbito

- Integridade de freeze e terminal
- Ciclo de vida de capacidades
- Validação executável
- Gates de release
- Suporte e distribuição v1.4
- Código do verificador de release

Os critérios de aceitação dependem do produto e do candidato atual. Estes exemplos não concedem autoridade nem substituem verificação real.

07 — Exemplo guiado: aplicação de gestão de um clube

ArquitecturaRepo · Manual completo de utilização · Edição v1.4 · Português

> Este é um **exemplo fictício de aprendizagem**, não uma aplicação implementada, um backend automaticamente gerado ou prova de que um modelo de IA entregou um CRUD aceite. Os nomes, dados e resultados abaixo são ilustrativos. Num produto real, requisitos, código, DB, runtime, permissões e evidência precisam vir do projeto adotado.

Neste capítulo

1. O produto que o clube pretende construir
2. Descobrir as questões que mudam o resultado
3. Instalar o controlo sem confundi-lo com a aplicação
4. Primeira capacidade: gestão de associados
5. Do Planner a I3: um pedido operacional
6. O que testar antes de aceitar
7. Segunda capacidade: mensalidades e pagamentos
8. Avaliar todo o exemplo
9. Primeiro ciclo com dados fictícios
10. Registar e encaminhar um defeito
11. As dependências do segundo ciclo
12. Exercício final de recuperação

1. O produto que o clube pretende construir

Imagine um clube local que utiliza folhas de cálculo para controlar associados, mensalidades, atividades e inscrições. Uma pessoa administrativa precisa saber quem está inscrito, quais quotas estão pagas e quantas vagas existem numa atividade. O objetivo é construir uma aplicação pequena, compreensível e confiável, que mantenha o histórico e evite erros de capacidade.

Para a primeira versão, o CLIENT decide que o programa será usado internamente. Não incluirá pagamentos online, uma aplicação móvel separada, faturação fiscal, notificações em massa ou integrações externas. Essas exclusões não são juízos definitivos contra as funcionalidades: são limites para a primeira entrega demonstrável, respeitando prazo, custo e complexidade.

Isso define a direção, mas não estabelece a linguagem, a base de dados, o hosting ou as permissões técnicas. O Planner precisa de confirmar o ambiente e a utilização; engenharia escolhe um stack proporcional aos riscos e ao código existente. O framework de coordenação não cria automaticamente formulários de associados nem grava dados por si só.

Uma sequência provável de capacidades é identidade dos associados, depois mensalidades e histórico, em seguida atividades/capacidade e finalmente inscrições com regras de elegibilidade. Essas capacidades dependem umas das outras. Não

convém registar pagamentos sem identidade estável, nem validar inscrição por quotas quando o estado de pagamento ainda não é confiável.

O objetivo da arquitetura é manter a continuidade dessas decisões durante o desenvolvimento, não criar quatro hierarquias de agentes. Para um clube pequeno, um Controller competente e um Planner claro podem ser suficientes. Especialistas ou revisores independentes entram quando existe risco e trabalho adequadamente delimitado.

2. Descobrir as questões que mudam o resultado

O Planner começa com discovery. Quantas pessoas utilizarão a aplicação? Há dois operadores simultâneos? O programa precisa funcionar sem internet? Como são feitos os backups? Que informações pessoais são armazenadas? Existe histórico que nunca pode ser apagado? Qual o prazo de uma primeira demonstração útil?

Uma regra proposta poderia ser: «Um administrador pode desativar um associado, mas os pagamentos históricos continuam acessíveis a pessoas autorizadas». O CLIENT precisa confirmar o significado da regra. Desativar não é necessariamente apagar; podem existir obrigações de retenção ou critérios específicos de consulta.

Outras perguntas: um associado em atraso pode inscrever-se numa atividade? E se tiver um acordo de pagamento? Uma inscrição cancelada liberta a vaga imediatamente? Existe lista de espera? Quem pode corrigir um pagamento? A data de vencimento depende do mês civil ou da data de adesão? Sem essas decisões, o programa pode apresentar respostas contraditórias em ecrãs diferentes.

Área	Comportamento proposto para a primeira versão
Associados	Criar, editar, pesquisar e desativar, preservando relações históricas.
Mensalidades	Registar pagamentos, identificar períodos em atraso e consultar histórico.
Atividades	Definir nome, horário, capacidade e vagas disponíveis.
Inscrições	Inscrever e cancelar com regras explícitas de capacidade e quotas.
Exclusões	Sem pagamentos online, app móvel, notificações em massa ou dashboards elaborados.

Cada linha continua provisória até confirmação do CLIENT. A necessidade de dados persistentes sugere o perfil `STATEFUL`, com controles de backup, currentness, migração e recuperação. Mas o perfil não cria uma política de pagamento nem confere permissão para guardar dados pessoais sem uma decisão adequada.

3. Instalar o controlo sem confundi-lo com a aplicação

Se o projeto do clube for novo e não tiver governação canónica anterior, é possível adicionar `.agentic-sdlc/` com um scaffold v1.4. Num checkout verificado do starter, o comando PowerShell ilustrativo é:

```
& "python" "tools\scaffold_project.py" --target "..\clube-app" --profile "STATEFUL" --
```

Esse comando cria a camada de controlo inicial no projeto de destino, com `HOLD / UNTRUSTED_CONTEXT`. Não demonstra que existe uma aplicação, não inicializa uma DB, não instala um LLM e não concede autoridade para executar uma campanha. Os passos de instalação do capítulo 01 mostram como inspecionar os resultados.

Se `clube-app/` já estiver governado, não se deve copiar um segundo Product State. Utilize o preflight somente leitura para governação existente e prepare a rota de adoção ou migração que preserva owners, fonte, DB e campanhas. O framework não é um atualizador in-place universal de controles de versões anteriores.

Depois do scaffold, Planner e engenharia investigam código, inventário, requisitos e estado existentes. Prepararam uma especificação de adoção revisada, fazem preflight, promovem apenas registos coerentes e demonstram reconstrução de contexto fresco. O CLIENT confirma escopo, prazo, restrições e prioridades antes de um charter de capacidade ser autorizado.

A decisão de stack deve considerar quem terá de manter a aplicação, ambiente real, conexão à rede, segurança e backups. Pode ser uma aplicação local simples ou um serviço mais elaborado se a necessidade justificar. Não se escolhe cloud, microserviços ou uma estrutura de agentes complexa apenas porque o framework admite esses conceitos.

4. Primeira capacidade: gestão de associados

Começar por associados resolve uma dependência fundamental. Quotas e inscrições precisam apontar para uma identidade estável. Uma capacidade inicial poderia definir:

> Um operador autorizado consegue criar um associado, consultar os seus dados, corrigi-los, pesquisá-lo e desativá-lo; pagamentos ou inscrições históricas permanecem preservados e disponíveis dentro das permissões previstas.

O charter identifica interfaces afetadas, ownership de dados, comportamento de erro e gates. Não precisa prescrever o nome de cada método. I3 organiza implementação, testes, correções, integração e revisão do candidato resultante. O Planner avalia a entrega contra os requisitos confirmados, não pela quantidade de código produzido.

Os testes relevantes abrangem identificador duplicado, campos inválidos, escrita e readback, persistência após reinício, filtros de ativos/inativos, histórico de relações, acesso não autorizado e interface real quando existe UI. A prova física deve executar o percurso do utilizador; uma unit test de validação não confirma que a ficha pode ser encontrada.

O CLIENT não precisa escolher um ORM, tipo de índice ou estrutura de diretórios. Engenharia pode decidir essas questões e explicar os trade-offs que alterem orçamento, acesso ou risco. Se o programa usar dados existentes, a migração deve respeitar o estado atual, backups e permissões; não se deve tratar o clube fictício deste manual como uma base real.

5. Do Planner a I3: um pedido operacional

Um dispatch completo pode continuar curto. Num projeto real teria de ser adaptado aos refs e requisitos canónicos, mas a estrutura é esta:

Goal: Entregar a capacidade de gestão de associados dentro do charter atual.

Context: Reconstruir requisitos confirmados, Product State, fonte exata, ambiente e estado atual da DB.

Constraints: Preservar histórico, respeitar permissões e orçamento; não introduzir processamento de quotas nem funcionalidades não aprovadas.

Execution gates: Exercitar fisicamente criação, pesquisa, edição, desativação e readback após reinício. Testar inputs inválidos, duplicados e caminhos pertinentes de segurança e concorrência. Rever independentemente o candidato integrado.

Done when: Apresentar ref exato, evidência física, resultados dos gates, achados do reviewer, limitações e terminal estratégico honesto.

I3 pode dividir o trabalho em execuções táticas, ajustar interfaces internas, depurar, retestar e recuperar interrupções. Se um duplo clique cria dois associados com identificador supostamente único, a falha pertence à mesma campanha. Não se abre uma segunda campanha apenas para resolver duplicados.

Quando há operações múltiplas ou custo material, o Controller pode definir uma work order com inventário, limites de execução, orçamento e condições de paragem. Uma unidade de execução não pode ampliar a intenção do CLIENT nem chamar um serviço pago sem autoridade. O charter continua a ser o limite do trabalho.

6. O que testar antes de aceitar

Considere um registo totalmente fictício `ASSOCIADO-001`. A tabela seguinte é um plano de teste, **não prova de que a aplicação existe ou que o teste foi realizado**:

Etapa	Ação	Resultado esperado
1	Abrir a aplicação como administrador.	O controlo de acesso real permite o papel correto.
2	Criar `ASSOCIADO-001` com dados sintéticos válidos.	Exatamente um registo aparece com identidade estável.
3	Pesquisar pelo identificador permitido.	Aparece a ficha correta sem expor informação não autorizada.
4	Alterar um contacto fictício.	A mudança é visível após guardar e reler.
5	Fechar e reabrir a aplicação.	O dado persiste pelo caminho real de DB/recovery.
6	Desativar o associado.	Não aparece como ativo, mas a identidade histórica mantém-se.
7	Consultar histórico com permissão adequada.	Pagamentos e inscrições anteriores continuam disponíveis.
8	Tentar um dado inválido ou duplicado.	A aplicação rejeita sem corromper estado.

Registe entrada, esperado, observado, candidato, data e ambiente em cada etapa. Um retorno HTTP 200 não prova que a escrita persistiu; deve ser conferido por leitura adequada. Se o estado precisa sobreviver a reinício, teste o reinício. Se dois operadores existem no escopo, exercite ações simultâneas ou intercaladas, conforme a arquitetura.

Não utilize dados reais de associados em screenshots, vídeos ou issues públicos. A avaliação de experiência deve usar dados fictícios e um ambiente representativo. Um gate perceptual feito por pessoa competente complementa, mas não substitui, as provas de DB, segurança e integração.

Quando a aplicação ainda não existe, o estado honesto desses passos é `NOT RUN`. Não os marque como PASS porque o manual descreve como deveriam funcionar. A finalidade do exercício é aprender que evidência será necessária, não produzir uma narrativa de sucesso sem implementação.

7. Segunda capacidade: mensalidades e pagamentos

Depois de aceitar estrategicamente a identidade de associados, o CLIENT pode pedir registo de mensalidades e consulta de atrasos. O Planner confirma períodos, vencimentos, pagamentos parciais, correções, permissões e tratamento de duplicados. O estado «em dia» deve derivar de uma regra consistente e dados atuais.

Uma decisão possível é: «Um pagamento registado por engano é revertido com uma entrada de auditoria, sem eliminar a transação original». O CLIENT confirma a política; o Planner documenta a regra e o risco; I3 implementa persistência, validações e recuperação. Uma segunda submissão da mesma operação não deve produzir efeito duplicado quando a regra exige idempotência.

Se a aplicação envolver dinheiro real, os gates de autorização, segurança, auditoria e dados precisam ser proporcionais às consequências. O framework não traz um processador de pagamentos e não autoriza ligar um serviço externo por iniciativa de um agente. O exemplo de quotas serve para estudar dados e regras, não para assumir operações financeiras publicadas.

Atividades e inscrições vêm depois, quando identidade e estado de mensalidades são confiáveis. Uma pessoa só pode inscrever-se respeitando lotação e elegibilidade. Se os dados upstream ainda forem provisórios, iniciar a inscrição completa arrisca implementar regras contraditórias; o Planner deve reconhecer a dependência antes de autorizar a campanha.

8. Avaliar todo o exemplo

Ao concluir este capítulo, deverá conseguir explicar o que o CLIENT decide, o que o Planner confirma, que trabalho I3 executa, como é identificada a fonte atual, porque a revisão independente tem de ver o candidato exato e como se obtém uma aceitação real. Também deve reconhecer que instalar o framework não é entregar a aplicação.

O objetivo não é decorar cada nome de ficheiro JSON. É saber qual superfície contém a verdade atual, quem é responsável por alterá-la e que resultado observável demonstra uma capacidade. Se o processo produz dezenas de documentos mas não consegue mostrar um percurso de associado, ainda não resolveu o problema do clube.

O exemplo aplica-se também a uma API CRUD, gestão de reservas, biblioteca, aplicação educativa ou sistema interno de uma empresa. Os riscos concretos mudam,

mas a separação entre intenção, autorização, implementação, evidência e aceitação continua útil.

A mesma análise ajuda a comparar uma abordagem estruturada com programação direta num chat. Um protótipo isolado pode avançar mais depressa com prompts diretos; um produto longo pode ganhar em continuidade, revisão e recuperação. A superioridade de um método tem de ser medida no trabalho real, não proclamada pelo manual.

9. Primeiro ciclo com dados fictícios

Para praticar, invente «Pessoa de Teste A» com contacto obviamente fictício. Decida com o CLIENT quais campos são obrigatórios e qual identificador deve ser único. O Planner registra os requisitos; engenharia escolhe o modelo de dados, validações e limites de transação.

Execute o percurso: criar, listar, pesquisar, editar, reiniciar, reler, desativar e consultar o histórico autorizado. Registe em cada etapa o resultado realmente observado. A captura do primeiro ecrã é apenas o começo da prova; não demonstra que uma alteração sobreviveu a reinício ou que uma pessoa sem permissão foi bloqueada.

Acrescente casos negativos: identificadores repetidos, campos inválidos, falha controlada da DB durante uma escrita e submissões duplicadas. O comportamento de erro deve ser claro e consistente com os requisitos confirmados. Não se aceita um estado contraditório silencioso como resultado normal.

Se vários operadores podem aceder simultaneamente, verifique situações de concorrência: duas inscrições para a última vaga, dois pagamentos da mesma obrigação, edição simultânea de um contacto. Testes sequenciais não provam automaticamente que o sistema é seguro nessas situações. Escolha os ensaios de acordo com o risco e o ambiente real.

O feedback do primeiro ciclo pode gerar pequenos ajustes de UX: posição dos campos, mensagem de erro, facilidade de encontrar uma ação. Se a alteração está dentro do envelope, I3 corrige; se acrescenta uma regra de negócio, o Planner confirma a decisão com o CLIENT. A observação humana e o resultado técnico pertencem a gates diferentes.

10. Registar e encaminhar um defeito

Suponha que «Pessoa de Teste A» é desativada e o histórico de pagamentos desaparece. Se a retenção estava confirmada, é um bug in-charter. Registe candidato, passos, dados sintéticos, resultado observado e gate afetado. O Planner devolve um falsificador consolidado a I3.

O Controller investiga relações de persistência, corrige a causa, executa testes de regressão e readback após reinício, integra novo candidato e obtém revisão independente aplicável. Não cria uma campanha paralela e não pede ao CLIENT para decidir como configurar uma foreign key. Essas decisões de engenharia pertencem ao seu mandato.

Agora o CLIENT pergunta se o associado inativo pode ser reativado com o mesmo número. Se a política anterior não definia isso, não é automaticamente um bug. O Planner pergunta qual o comportamento desejado, confirma impacto na identidade,

permissões e pagamentos e decide se cabe no charter ou exige revisão estratégica.

Não se inventa aceitação a partir da história fictícia. Se a aplicação não foi construída, o teste continua `NOT RUN`. Se o gate físico passou mas o CLIENT ainda não experimentou a experiência, o gate perceptual permanece aberto. Se a DB não foi verificada, não se afirma persistência apenas pelo sucesso visual.

11. As dependências do segundo ciclo

As mensalidades precisam de um identificador de associado estável e de regras claras sobre pagamentos parciais, vencimentos, correções e duplicados. O Planner utiliza a capacidade aceite como input atual; I3 implementa com interfaces e dados vigentes. Não se volta a uma versão antiga guardada apenas na memória do chat.

Antes da implementação de inscrições, confirme que o estado de mensalidades consumido pelas regras de elegibilidade realmente existe e é atual. Uma regra de atraso ainda indefinida pode levar a conflitos entre API e interface. O critical path não significa iniciar mais agentes, mas construir capacidades na ordem que permite testar as dependências.

Uma entrega incremental permite ao CLIENT provar primeiro a gestão de associados e só depois investir em quotas. A experiência pode revelar novos requisitos, como campos específicos do clube ou um processo de reativação. Registar a mudança cedo e atualizar downstream de forma consciente costuma ser mais barato que descobrir contradições depois de implementar tudo.

Evite acrescentar complexidade automática: microserviços, cloud, aplicações móveis, serviços de pagamento e políticas elaboradas de autorização precisam de justificação concreta. A qualidade não se mede pelo número de tecnologias usadas, mas pela utilidade, segurança proporcional e capacidade de manutenção do produto.

12. Exercício final de recuperação

Imagine que hoje o desenvolvimento para e amanhã entra um Planner que nunca leu a conversa anterior. O repositório e o runtime autorizado têm de revelar se a capacidade de associados foi aceite, qual candidato e evidência a sustentam, que decisões sobre quotas estão abertas, quem controla a campanha e qual próximo passo é permitido.

Se aparecerem duas campanhas ativas para o mesmo recurso, é necessário reconciliar autoridade. Se um relatório antigo diz `CURRENT`, mas o ref da fonte é diferente, a contradição deve ser investigada. Se falta testemunho de DB, o trabalho dependente fica em HOLD específico enquanto prosseguem verificações seguras.

Classifique os eventos observados: decisão nova do CLIENT, bug dentro do charter, falha de ambiente, autorização externa pendente ou invalidation estratégico. Cada tipo tem owner e rota. Um sistema confiável permite fazer esta distinção sem recriar o contexto por tentativa e erro.

O exercício está terminado quando consegue passar de intenção a comportamento demonstrável e recuperar a continuidade depois de uma interrupção. A prova de valor não é a existência de `.agentic-sdlc/` nem o número de agentes. É o produto que alguém usa e a evidência de currentness que outra pessoa consegue verificar.

Fim do exemplo: consulte o índice desta edição para regressar a um capítulo ou utilizar as referências técnicas.

Fontes e âmbito

- Exemplo sintético mínimo
- Fork e bootstrap
- Discovery, requisitos e entrega
- Limites do CLIENT
- Padrão de charter
- Início público v1.4
- Attachment de governação existente

Os exemplos de clube são didáticos. Nenhum deles demonstra um sistema instalado, uma capacidade aceite, um pagamento real ou uma integração externa provada.